Generalization and Regularization

Learning curves, regularization, and model selection

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Training loss does not measure generalization

Training loss is computed on the examples used to fit θ.

$$\min_{\theta} \mathcal{L}_{\text{train}}(\theta) \quad \neq \quad \min_{\theta} \mathcal{L}_{\text{new data}}(\theta)$$

Training loss

How well the model fits examples used during optimization.

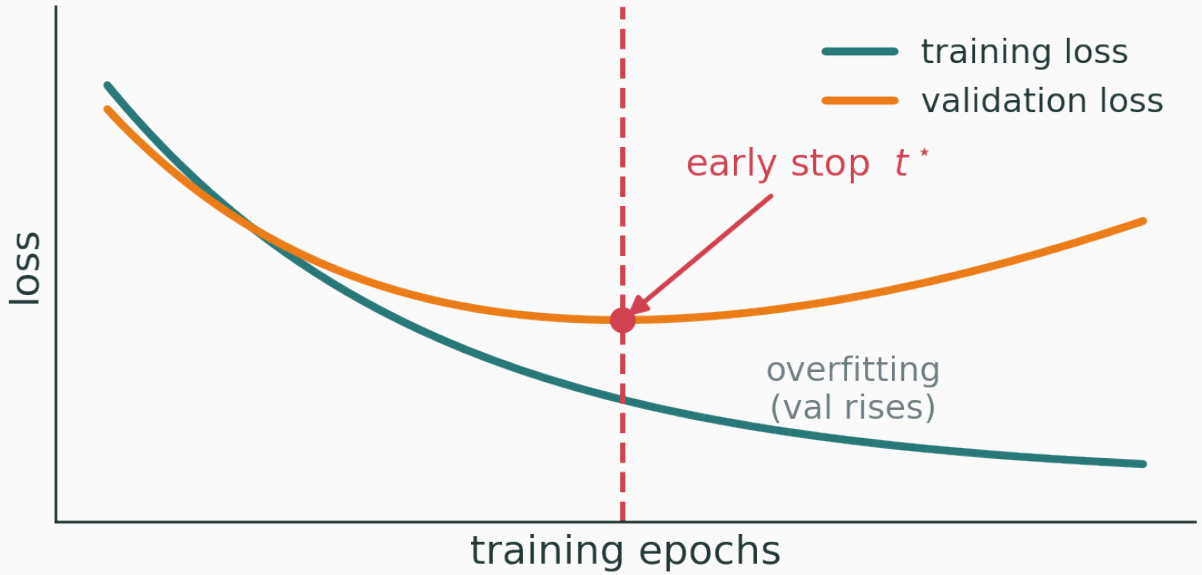
New-data loss

How well the learned rule transfers to unseen examples.

> Training loss can fall while new-data loss rises. Validation curves expose that gap.

Learning curves reveal overfitting

V VISUAL



$\mathcal{L}_{\text{train}} \downarrow$ throughout, while $\mathcal{L}_{\text{val}} \downarrow$ then $\uparrow$. Training continues, but generalization gets worse.

Train, validation, and test splits

D DERIVATION

split	job	allowed decisions
training	fit parameters θ	gradients; optimizer updates
validation	choose the recipe	λ, stopping time, p, augmentation, ε
test	estimate final generalization	**none.** Use once after the recipe is frozen.

> Using test accuracy to choose even one regularization knob turns the test set into another validation set. The running example keeps a separate dense test draw sealed until final evaluation.

Outline

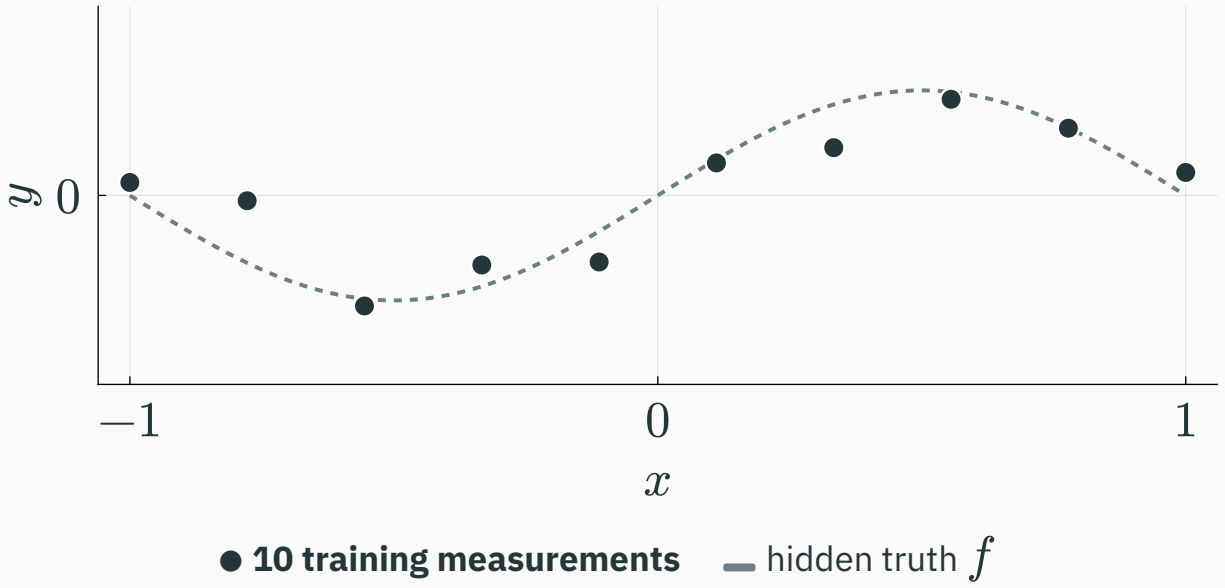
1. **Diagnose:** find the gap on a tiny dataset with a perfect training fit
2. **Weight decay:** penalize coefficient magnitude; tune its strength on validation
3. **Early stopping:** use training time as a capacity control
4. **Augmentation + input noise + mixing:** regularize in data space
5. **Dropout:** mask hidden activations during training; disable it at evaluation
6. **Label smoothing:** soften targets to limit confidence

Running example: degree-9 polynomial regression

V VISUAL

We measure a smooth signal at ten inputs; every reading carries noise:

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, 0.25^2).$$

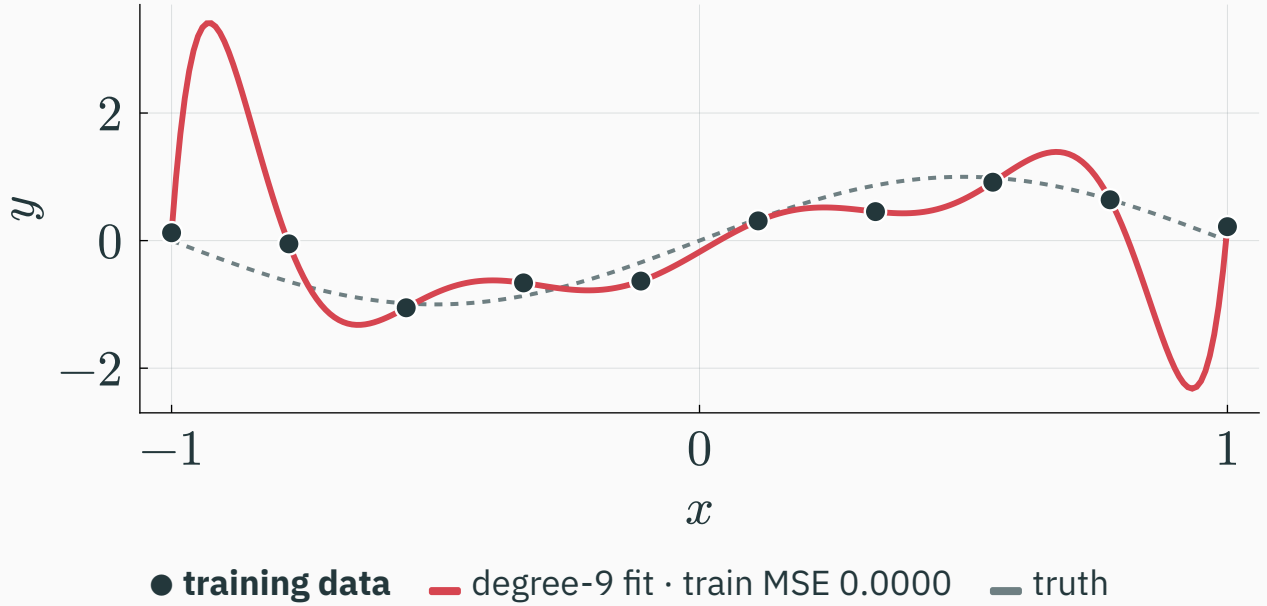


> From ES 335: expand features as $\varphi(x) = (1, x, x^2, ..., x^9)$ and fit $\hat{y} = \boldsymbol{w}^\top \varphi(x)$. This is a linear model with 10 weights and enough flexibility to bend between samples.

A degree-9 polynomial interpolates the training data

V VISUAL

Ten equations, ten weights: the least-squares fit passes through **every** point.

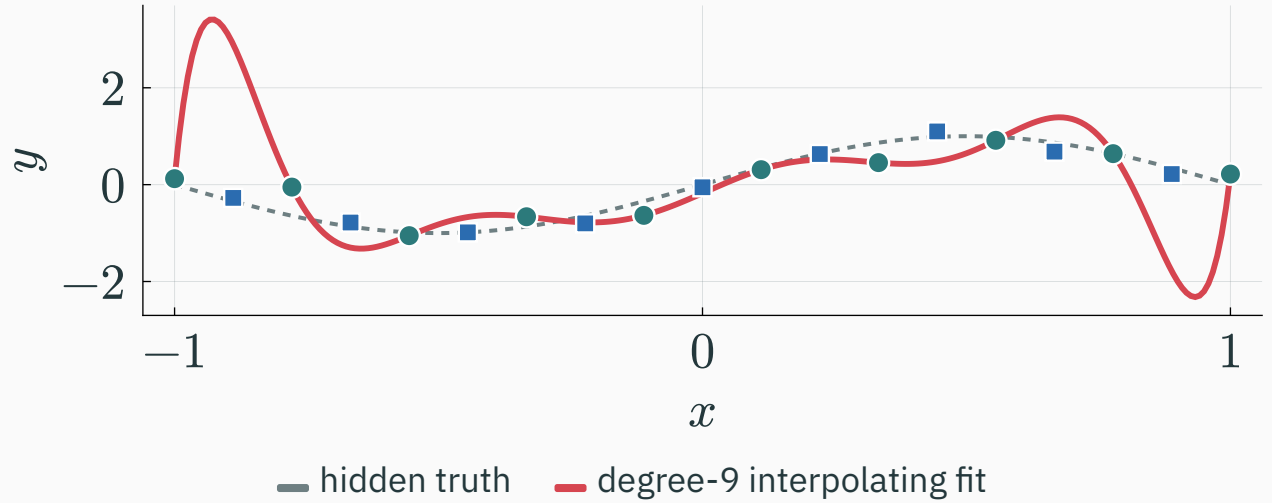


A training loss of 0.0000 shows interpolation of these points. It does not establish generalization.

Validation reveals poor interpolation between samples

V VISUAL

● **TRAIN: 10 measured points used to fit** ■ **VALIDATION: 9 unseen midpoints used only to evaluate**



Same fitted polynomial: **train MSE** 0.0000 **validation MSE** 1.70

> Even the true f would score $\approx \sigma^2 = 0.0625$ here. The interpolating polynomial scores $27\times$ worse on these unseen midpoints.

Diagnosing learning curves

D DERIVATION

Read the levels, gap, and trend of $(\mathcal{L}_{\text{train}}, \mathcal{L}_{\text{val}})$ together:

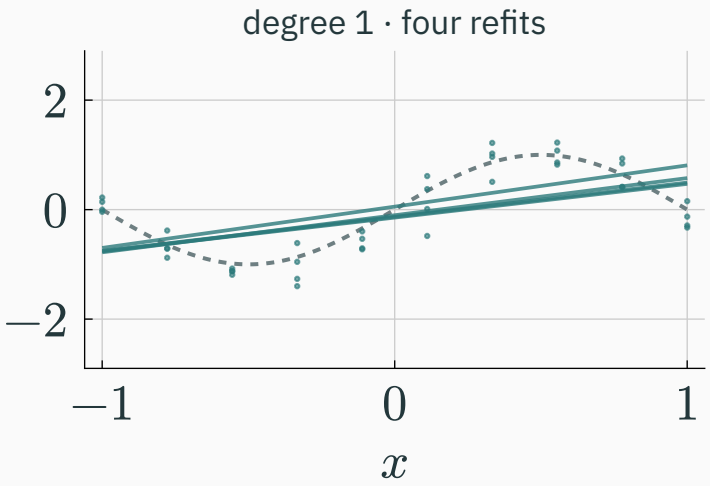
training	validation	diagnosis	first move
high	high, similar	underfit or broken optimization	repair fit (L5 to L6)
low	much higher	**overfitting**	**regularize**
low	low, similar	fit transfers to this split	run stress tests

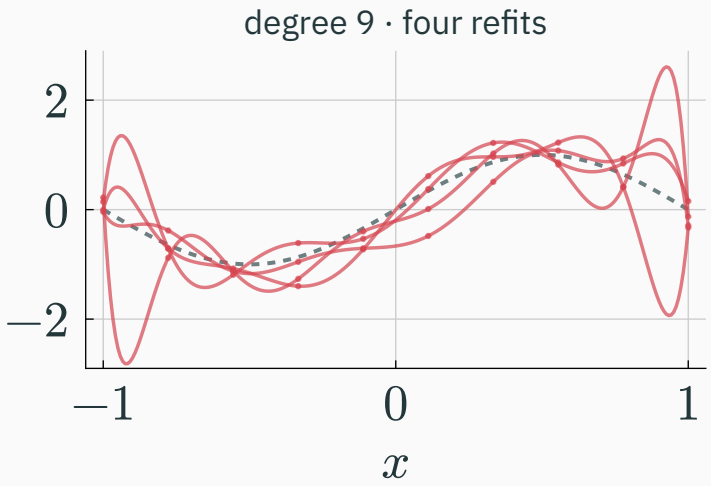
> A small gap is not enough; both losses can be high. The running example has train MSE 0.00 and validation MSE 1.70, which matches the second row.

Same x-locations, new noisy y-values, different fits

V VISUAL

Fixed design: keep the x_i, redraw ε_i, and get new $y_i = f(x_i) + \varepsilon_i$. The points move vertically.
A random-design experiment would redraw the x_i too.





Rigid model

stable across draws but wrong on average (**bias**)

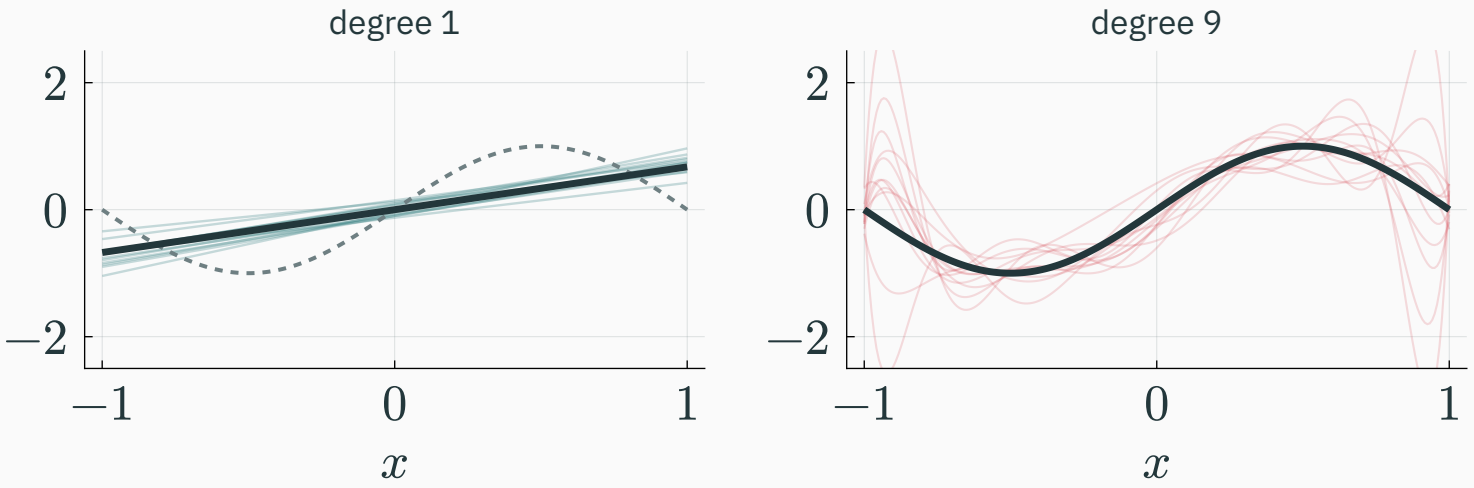
Flexible model

changes sharply across draws (**variance**)

Mean prediction across training sets

D DERIVATION

At each x, average the predictions from many training sets: $\mu_d(x) := \mathbb{E}_{\mathcal{D}}\left[\hat{f}_{d,\mathcal{D}}(x)\right]$.



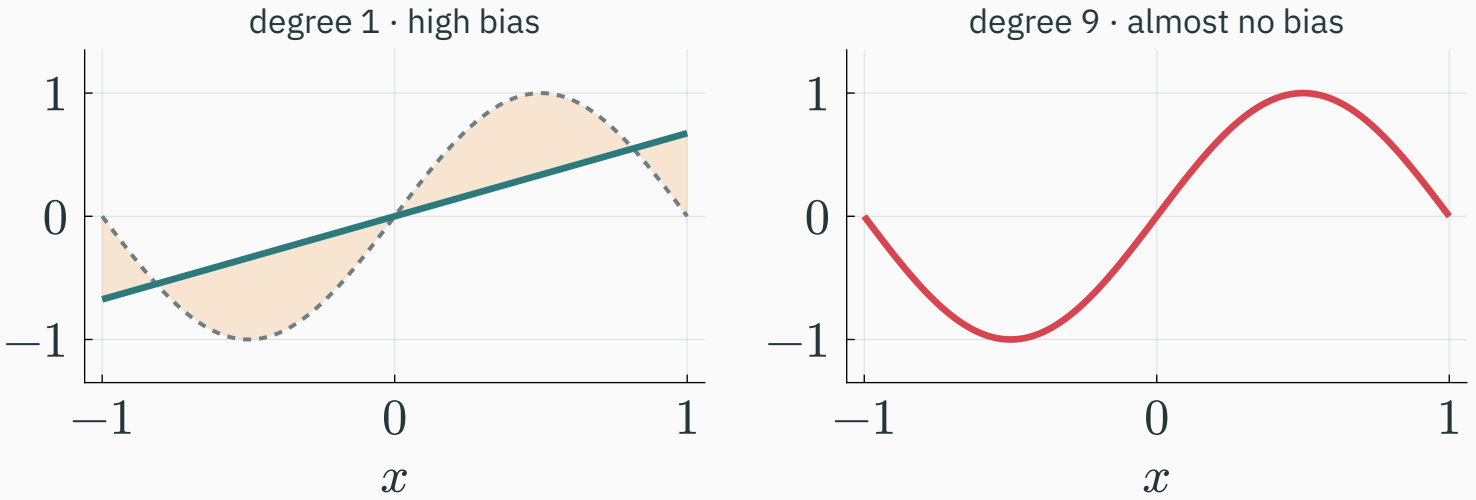
faint coloured curves = 12 possible fits — exact mean prediction μ_d dashed = truth f

Here $\mathcal{D}$ changes only through label noise at fixed x_i. The input locations stay fixed.

Bias is the error of the mean prediction

D DERIVATION

$$b_d(x) := \mu_d(x) - f(x), \quad \text{and squared bias} = (b_d(x))^2.$$



degree 1 · mean b^2 at the 9 validation midpoints: **0.212**

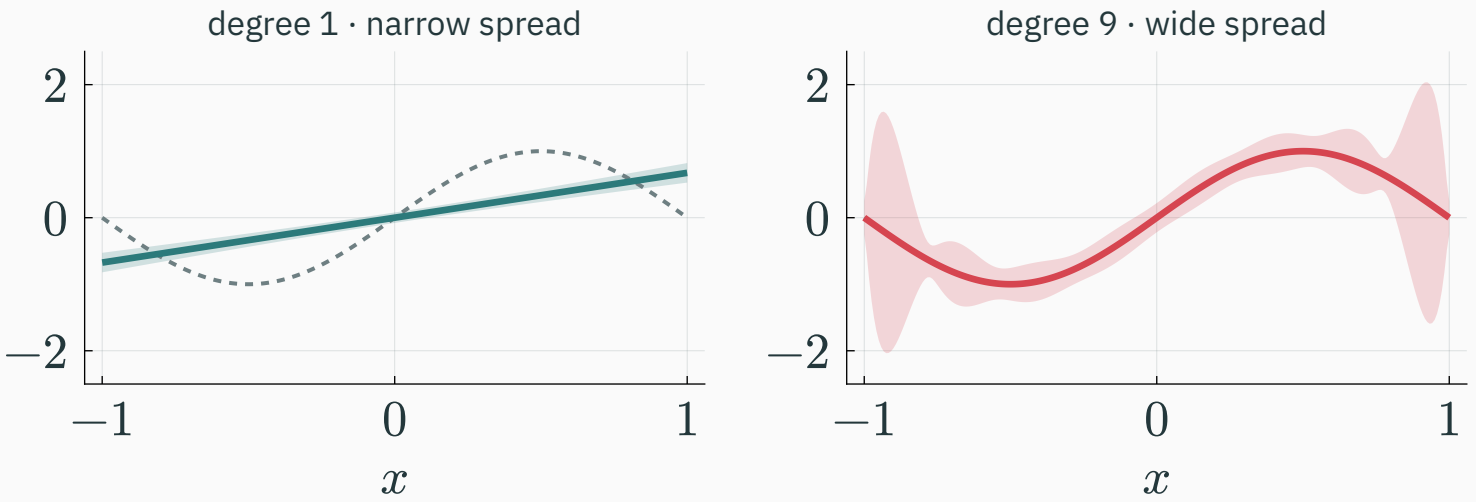
degree 9 · mean b^2: 6.6×10^{-10}

Orange shading shows signed bias. MSE uses squared bias. The tiny degree-9 bias is specific to this smooth signal and fixed design.

Variance measures sensitivity to label noise

D DERIVATION

$$\mathrm{Var}_{\mathcal{D}}\left[\hat{f}_d(x)\right] := \mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_d(x) - \mu_d(x)\right)^2\right].$$



degree 1 · mean variance at the 9 validation midpoints: **0.011**

degree 9 · mean variance: **0.606**

Shading shows $\mu_d(x) \pm 1$ SD across training sets. It is not a prediction interval for a new noisy target.

Expected test MSE has three components

D DERIVATION

For a fresh target $Y = f(x) + \varepsilon_{\text{test}}$, with independent zero-mean noise:

$$\mathbb{E}_{\mathcal{D},\varepsilon_{\text{test}}}\left[\left(\hat{f}_d(x) - Y\right)^2\right] = \underbrace{\left(b_d(x)\right)^2}_{\text{squared bias}} + \underbrace{\text{Var}_{\mathcal{D}}\left[\hat{f}_d(x)\right]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

model	**mean bias²**	**mean variance**	noise	expected MSE
degree 1	0.212	0.011	0.063	0.285
degree 9	≈ 0	0.606	0.063	0.669

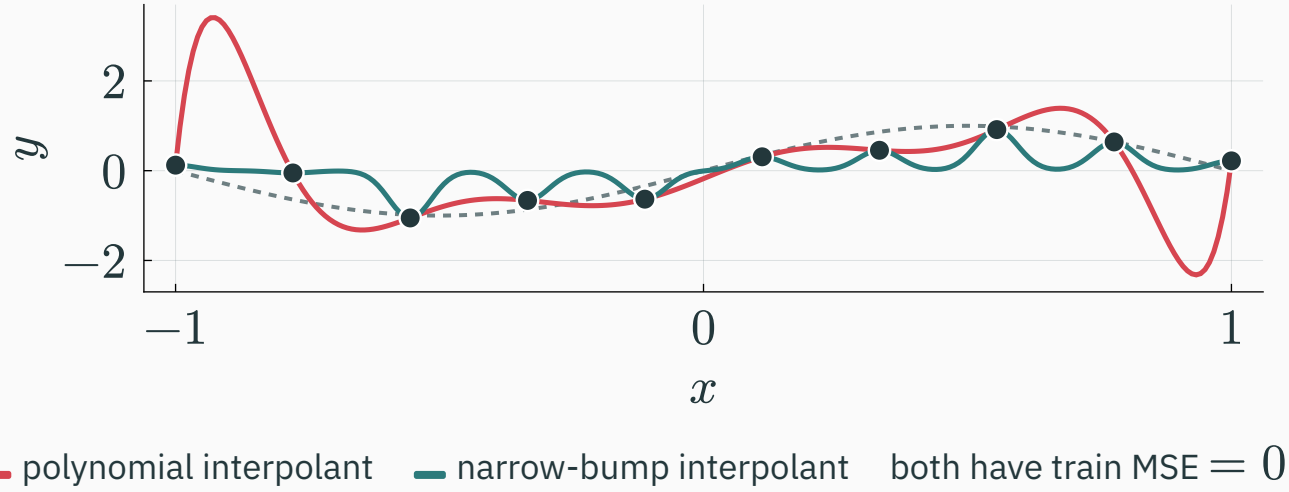
These are exact expectations at the same nine validation midpoints. Cross-terms vanish under independent zero-mean noise. The observed MSE 1.70 is one realization.

The degree-9 fit removes bias here, but its high variance raises the expected test error.

Many functions can interpolate the training data

V VISUAL

Narrow radial bumps through the same ten points **also** score 0.0000:



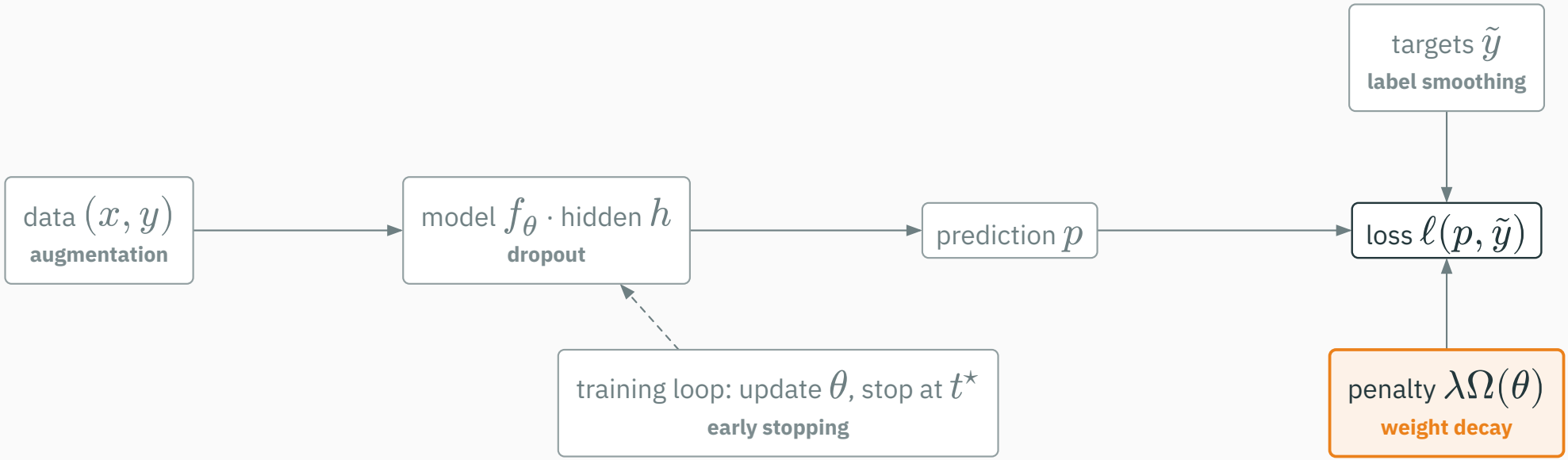
Both fits agree at the 10 training points but make different predictions between them.

> Large neural networks can memorize randomly shuffled labels (Zhang et al., 2017). Zero training error shows **capacity** but says nothing about generalization. Architecture, optimization, augmentation, weight decay, and stopping determine which zero-error predictor training finds.

Weight decay changes the objective

V VISUAL

Regularizers differ mainly in where they enter training. Weight decay adds a preference for smaller parameters to the objective:



Make large weights costly, steering training away from coefficient-heavy fits that match noise; validation chooses λ.

L_2 regularization adds a norm penalty

D DERIVATION

$$\mathcal{L}_{\text{data}(w)} = \frac{1}{2n}\|Xw - y\|_2^2, \quad \mathcal{J}(w) = \underbrace{\mathcal{L}_{\text{data}(w)}}_{\textbf{fit}} + \underbrace{\frac{\lambda_{\text{wd}}}{2}\|w\|_2^2}_{\textbf{shrink}}$$

Convention: the $\frac{1}{2}$ factors make $\nabla\mathcal{J} = \frac{X^\top(Xw-y)}{n} + \lambda_{\text{wd}} w$. Lecture 1 derived the Gaussian-prior view. Here we examine how the same term changes the fitted function and the optimizer update. Revisit: interactive-articles/map-regularization

The interpolating polynomial has large coefficients

V VISUAL

The zero-training-loss polynomial uses large coefficients that cancel each other:

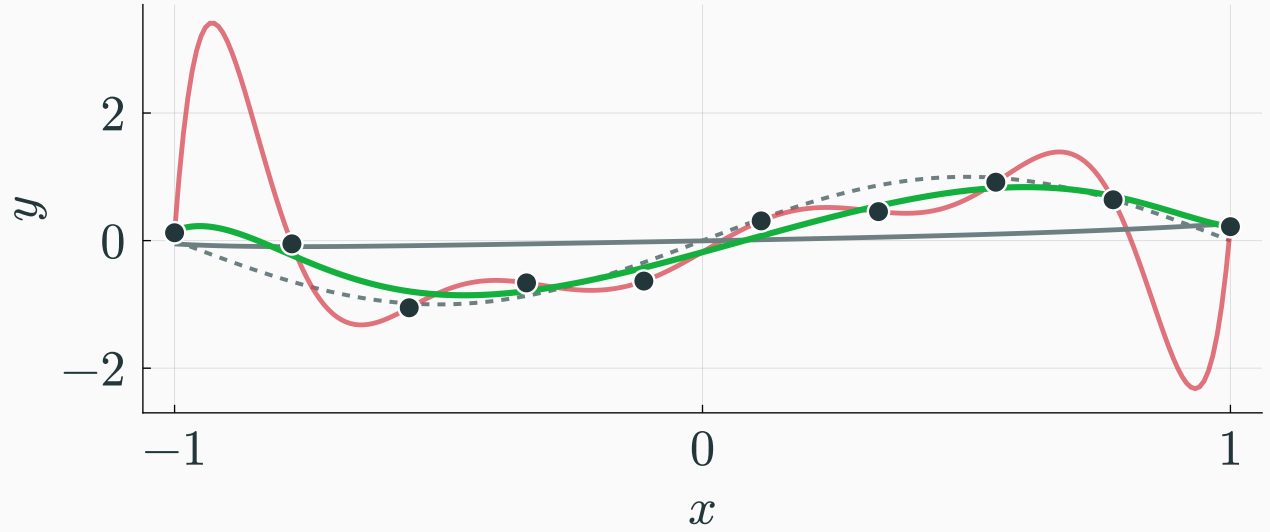
Degree-9 interpolant: largest coefficient magnitude $= 362$; $\|w\|_2 = 465$.

In this polynomial basis, the oscillations require large coefficients, so an L_2 penalty assigns that fit a large cost.

λ controls the strength of weight decay

V VISUAL

The data and features stay fixed. The legend reads: chosen $\lambda \to$ resulting $\|w\|_2$.



$\lambda = 0 \to \|w\|_2 = 465$ $\lambda = 10^{-3} \to \|w\|_2 = 3.9$ $\lambda = 1 \to \|w\|_2 = 0.2$

Increasing λ trades data fit against coefficient size. At $\lambda = 0$ the model interpolates; too much shrinkage underfits.

Checkpoint: which metric should choose λ?

Q QUESTION

The sweep below is computed from the running example:

λ	train MSE	val MSE	$\|w\|_2$
10^{-6}	0.014	0.151	35.1
10^{-4}	0.018	0.077	5.6
10^{-3}	0.023	0.065	3.9
10^{-2}	0.062	0.111	1.8
10^{0}	0.286	0.405	0.2

Which column is allowed to choose λ?

A. train MSE, because it is the objective we optimize

B. validation MSE

C. $\|w\|_2$, because smaller seems safer

D. none; average all three columns

Answer: choose λ on validation data

A ANSWER

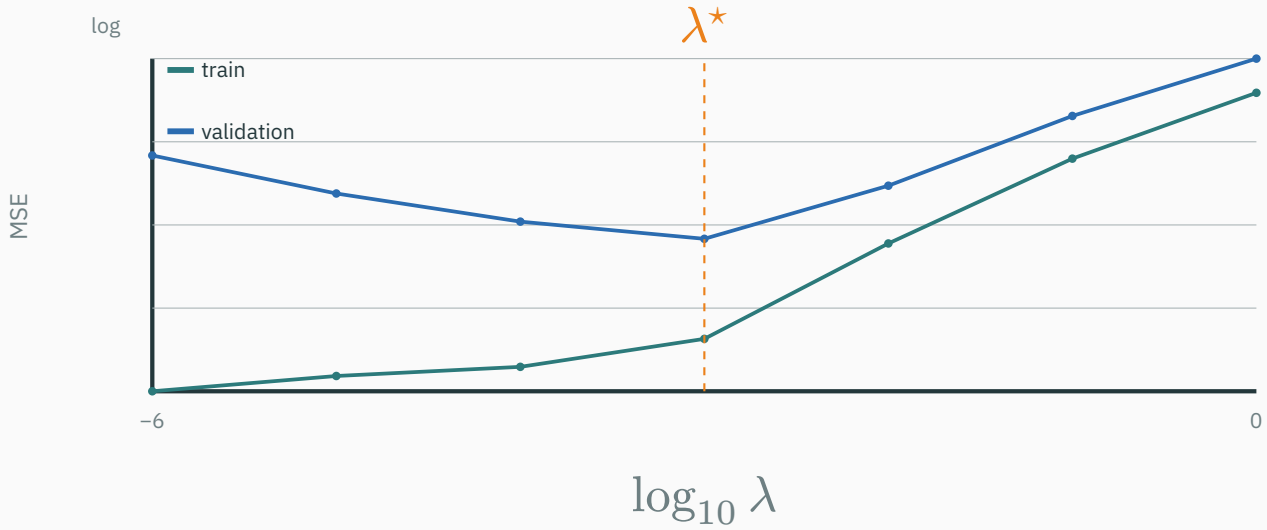
Correct: B — validation MSE picks $\lambda^\star = 10^{-3}$

Why: in this ridge sweep the **unpenalized** training MSE is minimized at $\lambda = 0$; the weight norm is a mechanism check, not a target.

Use the same procedure for every regularizer: sweep the knob and select by validation.

Sweep λ and choose the lowest validation loss

V VISUAL



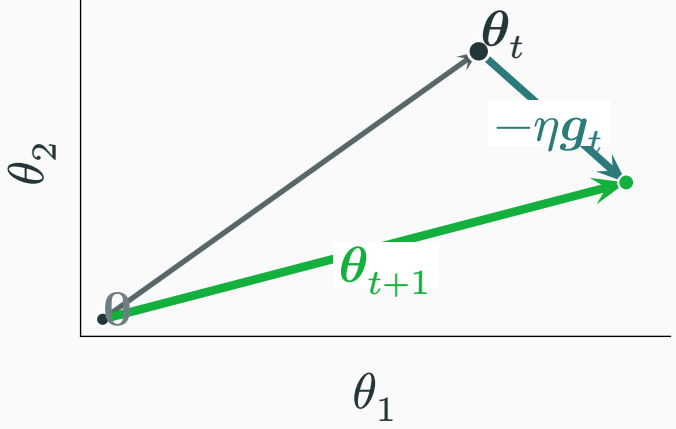
> Too little weight decay leaves a high-variance fit; too much makes the model underfit. Search λ on a log scale and keep the value with the lowest validation loss. Real curves may be noisy or flat.

Weight decay shrinks weights before the data update

V VISUAL

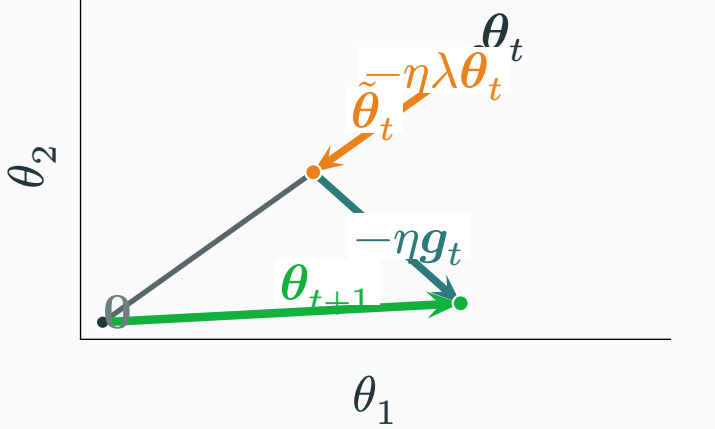
Plain SGD

$$\theta_{t+1} = \theta_t - \eta g_t$$



L_2-regularized SGD

$$\tilde{\theta}_t = (1 - \eta\lambda)\theta_t, \quad \theta_{t+1} = \tilde{\theta}_t - \eta g_t$$



Orange retraces grey toward 0. The same **teal step** then turns sharply. **Green is the result**.

The regularized update first shrinks θ toward 0, then applies the usual data-gradient move.

Deriving the L_2-SGD update

D DERIVATION

The penalty's gradient is $\lambda\boldsymbol{\theta}_t$, so the vector update factors:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta(\boldsymbol{g}_t + \lambda\boldsymbol{\theta}_t) = \underbrace{(1-\eta\lambda)\boldsymbol{\theta}_t}_{\text{shrink first}} - \underbrace{\eta\boldsymbol{g}_t}_{\text{then fit}}$$

For one coordinate, let $\theta_t = 5$, $g_t = 3$, $\eta = 0.1$, and $\lambda = 0.2$:

piece	substitution	value
shrink	$(1 - 0.1 \cdot 0.2) \times 5 = 0.98 \times 5$	4.90
fit	-0.1×3	-0.30
θ_{t+1}	$4.90 - 0.30$	4.60

Each coordinate is multiplied by 0.98, then the data gradient contributes -0.30.

L_2 regularization with PyTorch SGD

With `momentum=0`, `weight_decay=λ` is the coupled L_2 penalty from the previous slide:

$$p \leftarrow p - \mathrm{lr}\ (p.\mathrm{grad} + \mathrm{weight_decay} \cdot p)$$

Reproduce the $5.00 \rightarrow 4.60$ step

```
w = nn.Parameter(torch.tensor([5.0]))
opt = torch.optim.SGD(
    [w], lr=0.1, momentum=0.0,
    weight_decay=0.2,
)

w.grad = torch.tensor([3.0])  # data gradient
opt.step()
w.item()                      # 4.60
```

Use it in the training loop

```
opt = torch.optim.SGD(
    model.parameters(), lr=1e-2,
    momentum=0.0, weight_decay=1e-4,
)

for x, y in train_loader:
    opt.zero_grad(set_to_none=True)
    loss = criterion(model(x), y)  # data loss
    loss.backward()
    opt.step()                     # uses grad + λp
```

> Use either optimizer `weight_decay` or add $(\lambda/2)\|\theta\|_2^2$ to the loss. Do not use both. With optimizer-side decay, `loss.item()` omits the penalty. Every parameter in that group decays. PyTorch SGD API

L_1 regularization and sparsity

V VISUAL

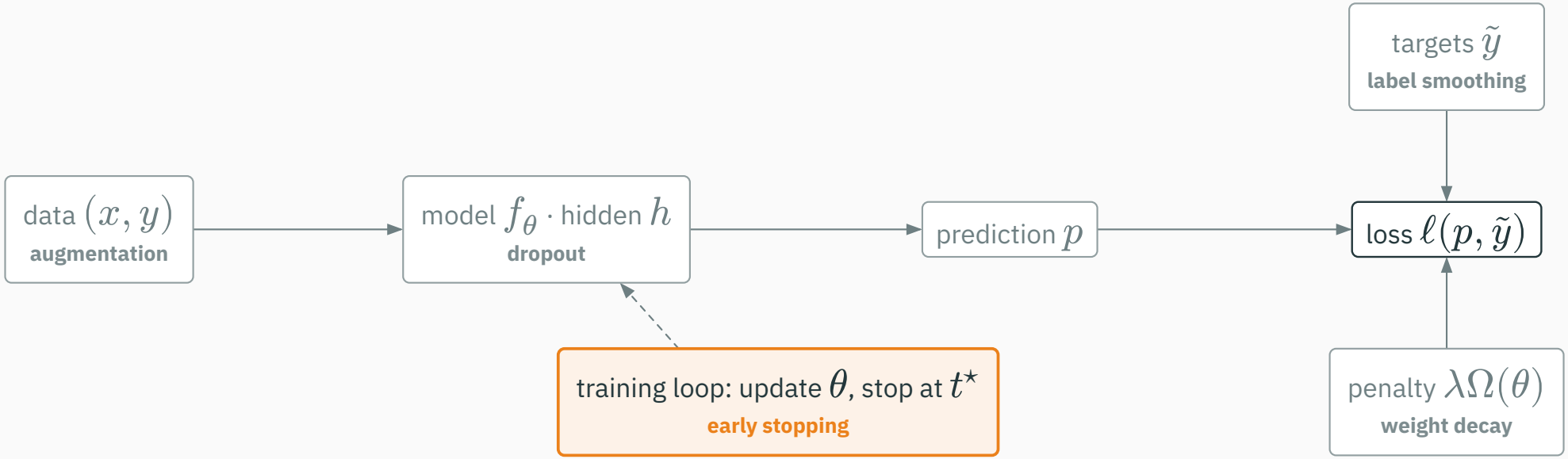
penalty	gradient force	pressure	role in DL
$\lambda\|\theta\|_2^2/2$	$\lambda\theta$ (proportional)	shrink everything a little	default
$\lambda\|\theta\|_1$	$\lambda\ \mathrm{sign}(\theta)$ (subgradient)	promote sparse solutions	specialized

> From Lecture 1: L_2 is a Gaussian prior, L_1 a Laplace prior. Ordinary SGD need not land exactly on zero; proximal/thresholding methods or explicit pruning create exact zeros. In standard dense networks, L_1 is secondary.

Interactive: adjust λ at interactive-articles/weight-decay

Early stopping changes the training loop

V VISUAL

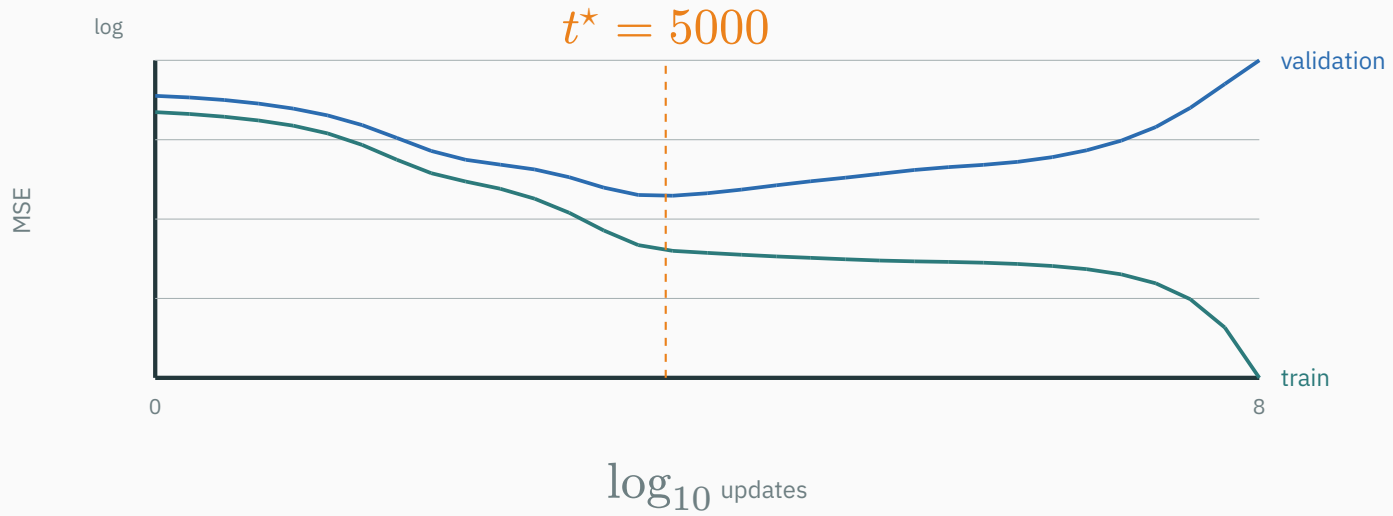


Keep the validation winner before later updates overfit the training set; patience decides when to stop.

Watch training and validation loss together

V VISUAL

Same model, **no penalty**. Start gradient descent from $w = 0$ and log both losses:



Training loss keeps falling. Validation loss reaches its minimum at $t^\star$ and then rises, giving a U-shaped curve against training time.

Keep the checkpoint with lowest validation loss

D DERIVATION

$$t^\star = \arg\min_t \mathcal{L}_{\text{val}}(\boldsymbol{w}_t)$$

checkpoint t	**train MSE**	**val MSE**	verdict
100	0.097	0.153	discard
1000	0.043	0.089	discard
5000	0.020	0.061	**restore this one**
30000	0.018	0.075	discard
1000000	0.016	0.115	discard

Patience decides when to stop

V VISUAL

Example: `patience = 3`, `min_delta = 0`:

epoch	20	21	22	23
validation loss	**0.570**	0.575	0.574	0.580
`wait`	0	1	2	**3**
action	**save**	continue	continue	**stop**

> `patience = P` allows P consecutive validation checks without enough improvement. `min_delta` sets the improvement required to reset `wait`. A positive value ignores tiny fluctuations.

Stop after epoch 23 and restore epoch 20. Patience controls when training stops; validation loss still decides which checkpoint to keep.

PyTorch: save, wait, restore

V VISUAL

```python
from copy import deepcopy
patience, min_delta = 5, 0.0
best_val, bad_epochs, best_state = float("inf"), 0, None

for epoch in range(max_epochs):
    train_one_epoch(model, train_loader, optimizer)
    val_loss = evaluate(model, val_loader)

    if val_loss < best_val - min_delta:
        best_val, bad_epochs = val_loss, 0
        best_state = deepcopy(model.state_dict())
    else:
        bad_epochs += 1
        if bad_epochs >= patience:
            break

model.load_state_dict(best_state)  # restore; do not keep the final epoch
```

> `patience = 5` allows five consecutive validation checks without sufficient improvement. `min_delta = 0` keeps the strict $\arg\min$. Use a positive value when tiny gains should not reset the clock.

Why validation loss eventually rises

V VISUAL

training stage	what the model is fitting	what the losses do
earlier epochs	patterns repeated across many examples, which are useful on new data	**train** ↓ **validation** ↓
later epochs	training-specific quirks, rare cases, and noise	**train** ↓ **validation** ↑

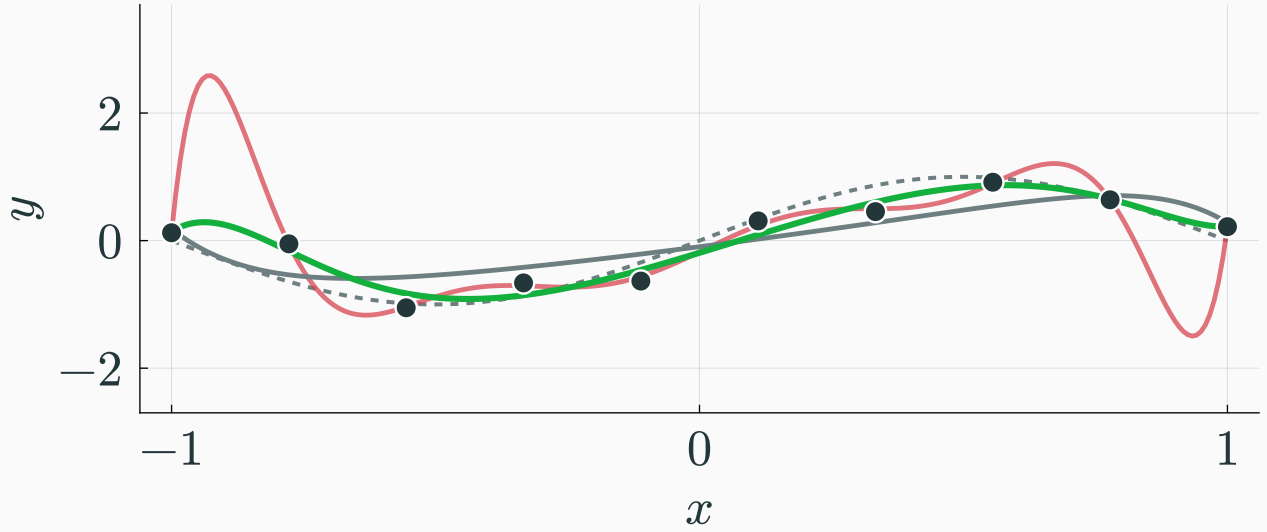
> This ordering is common but not guaranteed. The validation curve shows whether more training still helps on unseen data.

When validation loss starts to rise, restore the checkpoint from its minimum.

Early stopping limits what the model can fit

V VISUAL

Three checkpoints from the **same run**, with no penalty:



$t = 100$ · still underfitting $t^\star = 5000$ · kept $t = 10^8$ · fits the noise

At the chosen stopping time, the model has not yet fitted the weak directions that carry much of the noise.

Exercise: predict the learning curves

I INTERACTIVE

INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L07/01_overfitting_and_early_stopping.ipynb

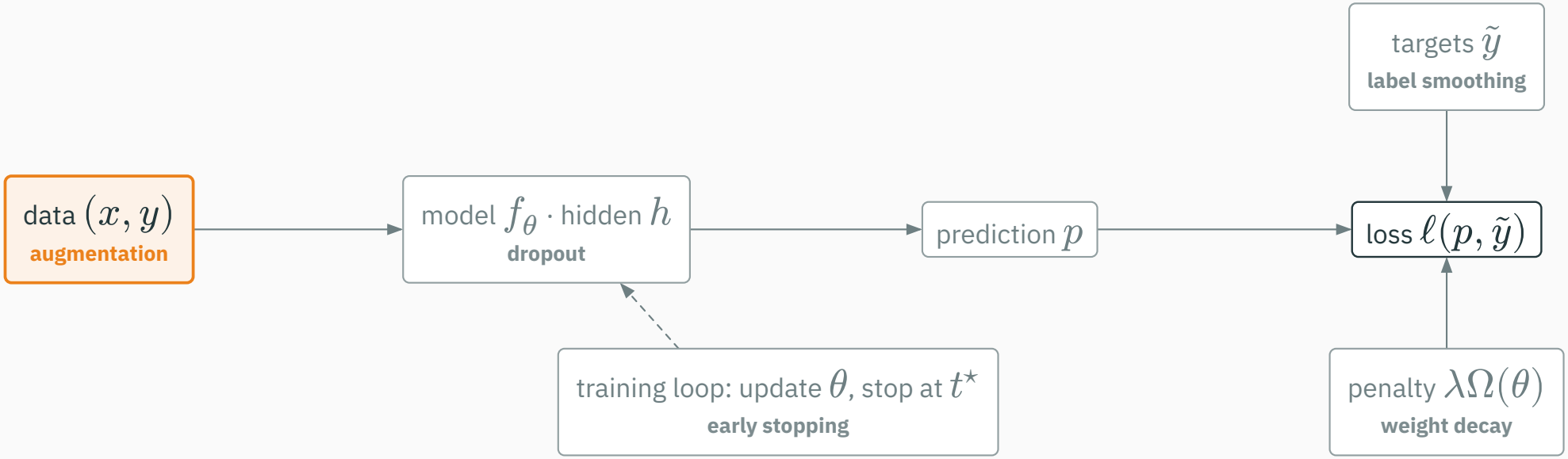
In this NumPy lab, a flexible model overfits a small dataset. Log both curves and restore the best checkpoint.

Before executing each cell, write down:

- the epoch range containing the validation minimum;
- a **patience** value that would catch it;
- how doubling the noise changes test error at the restored checkpoint.

Data augmentation changes the training examples

V VISUAL



Show task-valid versions of each sample, so the model cannot rely on accidental details of one stored form; update targets when needed.

TorchVision: original image and random crop

V VISUAL

```
from torchvision.transforms import v2
```

original pixels

```
v2.Identity()
```

one crop draw

```
v2.RandomResizedCrop(512, scale=(.45, .45))
```

> Cropping changes position and scale. The label remains valid only while the object stays visible.

TorchVision: horizontal flip and colour jitter

V VISUAL

```
from torchvision.transforms import v2
```

one flip draw

```
v2.RandomHorizontalFlip(p=.5)
```

one colour-jitter draw

```
v2.ColorJitter(brightness=.4, saturation=.6, hue=.2)
```

> A flip assumes left and right do not matter. Colour jitter assumes that illumination and tint do not matter.

TorchVision: blur and rotation

V VISUAL

```
from torchvision.transforms import v2
```

fixed blur

```
v2.GaussianBlur(21, sigma=4.)
```

one rotation draw

```
v2.RandomRotation(20, fill=(247, 245, 240))
```

> Start with mild transforms, tune them on validation data, and inspect a transformed batch.

Each panel was rendered by the TorchVision v2 call shown above.

One stored example becomes many valid training views

V VISUAL

Augmentation asks for the right answer across a family of task-valid views.

STORED EXAMPLE
(x_i, y_i)
one file and its target

→

FRESH VALID VIEW
$\tau_i \sim \mathcal{T}$
crop, flip, jitter, ...

→

TRAIN ON THIS VIEW
$(x_i', y_i') = \tau_{i(x_i, y_i)}$
a new draw on the next load

> The files do not multiply. The **stream of views** does: loading the same item again draws a new τ_i.

For classification, usually $y_i' = y_i$. For detection and segmentation, move boxes or masks with the image.

A minibatch samples one fresh view per item

D DERIVATION

Draw independently for each item, compute its transformed loss, then average.

FOR EACH $i = 1, \ldots, B$

$$(x_i, y_i) \sim \mathcal{D}_{\text{train}}$$
$$\tau_i \sim \mathcal{T}$$
$$(x_i', y_i') = \tau_{i(x_i, y_i)}$$

AVERAGE THE B LOSSES

$$\hat{\mathcal{L}}_B = \frac{1}{B} \sum_{i=1}^{B} \ell\left(f_{\theta(x_i')}, y_i'\right)$$

Each term uses one randomly transformed view of one minibatch item.

> This random minibatch loss is the **Monte Carlo estimate** used by SGD. A new batch produces a new estimate.

Repeated minibatches estimate the augmented objective

D DERIVATION

Average over training examples and over valid transformations.

$$\mathbb{E}\left[\hat{\mathcal{L}}_B\right] = \mathcal{L}_{\mathrm{aug}(\theta)}$$

$$\mathcal{L}_{\mathrm{aug}(\theta)} = \mathbb{E}_{(x,y)\sim\mathcal{D}_{\mathrm{train}}}\ \mathbb{E}_{\tau\sim\mathcal{T}}\ \left[\ell\left(f_{\theta(x')}, y'\right)\right]$$

$$(x', y') = \tau(x, y)$$

INNER AVERAGE

For one example, average its loss over task-valid views $\tau \sim \mathcal{T}$.

OUTER AVERAGE

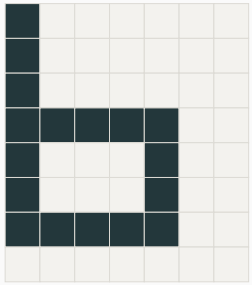
Then average across examples from the training distribution.

We cannot enumerate every valid view. Repeated random minibatches approximate both averages.

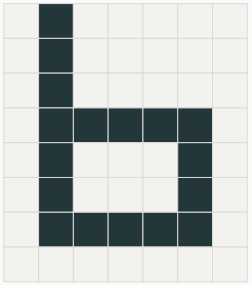
Check whether each transform preserves the label

V VISUAL

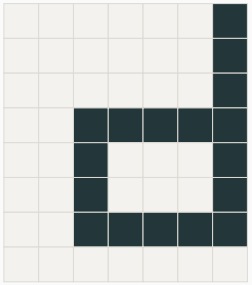
For pixel letters with classes $\{a, b, c, d, e\}$, check what each transform claims:



original, label **“b”**



shifted, still **“b”**



flipped, now reads **“d”**

Horizontal flip preserves flower classes, but it can change a letter’s class.

Checkpoint: valid transformations for letters

Q QUESTION

Which transformation preserves the label for every letter?

A. horizontal flip, because left/right orientation should not affect the class

B. rotate by $180°$, because orientation should not affect the class

C. shift by one or two pixels, keeping the glyph in frame

D. any transform that makes training loss go up

> Choose an answer, then give a counterexample for each rejected option.

Answer: only the shift preserves every label

A ANSWER

Correct: C — small shifts keep the glyph and the label

Why: flip turns $b \leftrightarrow d$; a $180°$ rotation turns 6 into 9; "harder training" is not a label-preservation argument.

transform	what the task must ignore	when the label breaks
random crop	position and scale (a little)	the crop removes the object
horizontal flip	left-right orientation	text, digits, anatomy (the heart has a side)
colour jitter	illumination and tint	the label depends on colour

For detection, geometric transforms must also move boxes and masks. Transform the **supervision** with the image.

Specify what happens to the target

D DERIVATION

$$A_{\tau(x,y)} = \left(g_{\tau(x)}, h_{\tau(y)}\right)$$

One sampled transform τ determines both the input map and the supervision map.

KEEP TARGET $h_{\tau(y)} = y$. The target is invariant, as with an object class under a safe crop.

UPDATE TARGET $h_{\tau(y)} \neq y$, but the update is known. Coordinates move, timestamps shift, or a signed value flips.

REJECT OR RELABEL There is no trustworthy h_τ. The transform changes the meaning or removes information needed to determine the target.

The task defines h_τ. An image library cannot infer what the target means.

The same flip requires different target updates

V VISUAL

as stored, travelling **right**

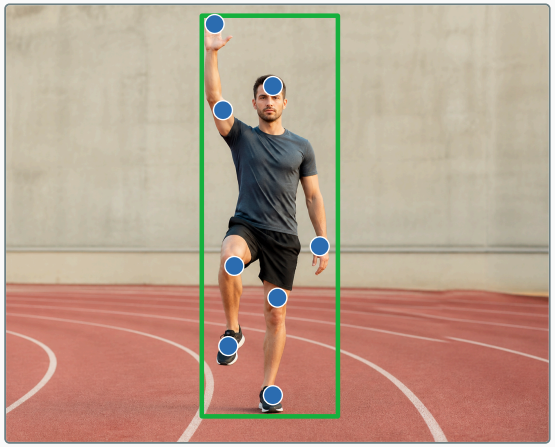
same pixels flipped, travelling **left**

task	target before	target after the same flip
object class	cyclist	**cyclist**, unchanged
travel direction	right	**left**, changed
detection box	$[x_1, y_1, x_2, y_2]$	$[W - x_2, y_1, W - x_1, y_2]$, moved

The pixel operation is identical. The prediction task determines how the target changes.

Flip boxes and keypoints with the image

V VISUAL



course-generated source with exact overlays added in the deck

Boxes with half-open XYXY coordinates

$$[x_1, y_1, x_2, y_2] \rightarrow [W - x_2, y_1, W - x_1, y_2]$$

Crop or rotate: transform the corners, clip to the canvas, then drop invalid boxes **and their matching labels**.

Keypoints

$$p_j{}' \propto H_\tau p_{\pi_\tau(j)}$$

H_τ moves coordinates; π_τ swaps semantic left/right indices when a flip requires it. Update visibility when a point leaves the frame.

> A geometric transform applied only to the image produces incorrect supervision.

For sentiment, preserve meaning and polarity

V VISUAL

A paraphrase may keep the label. A negation changes it.

“The movie was fantastic!” POSITIVE

↙ ↘

KEEP LABEL · paraphrase

“The film was wonderful!”

POSITIVE

CHANGE LABEL · negation

“The movie was not fantastic.”

NEGATIVE

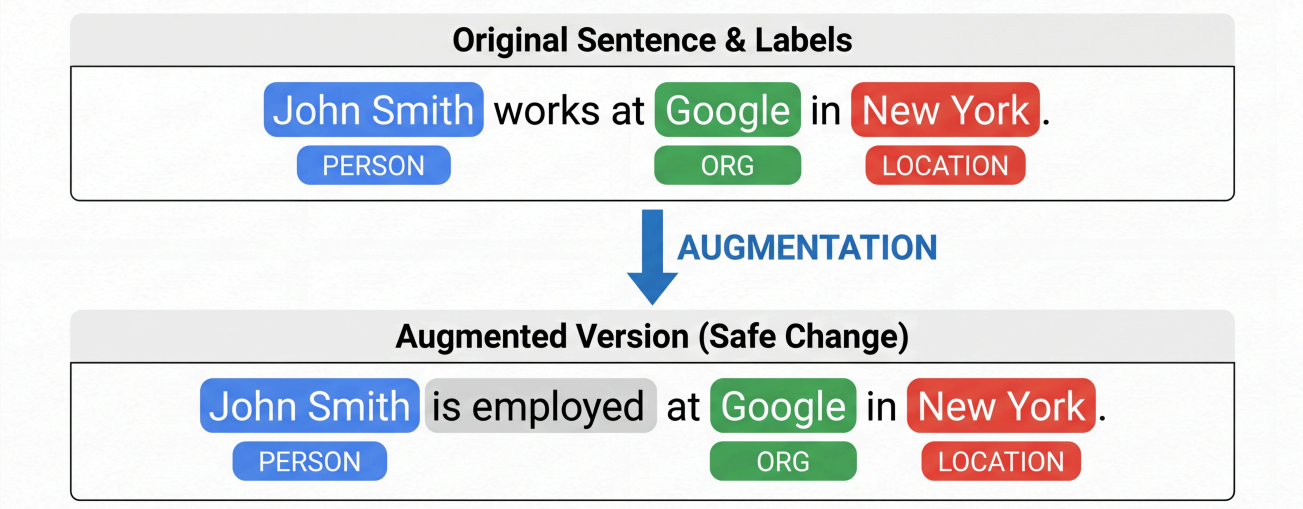
> The transform is valid only if the target is still correct. Otherwise relabel it—or reject it.

Example adapted from STT-AI Week 5: text augmentation.

For NER, edits can move the labelled spans

V VISUAL

Keep the entity text and type; recompute where each span begins and ends.



$$\mathcal{A} = \{(s_i, e_i, c_i)\} \rightarrow \mathcal{A}' = \{(s_i', e_i', c_i)\}$$

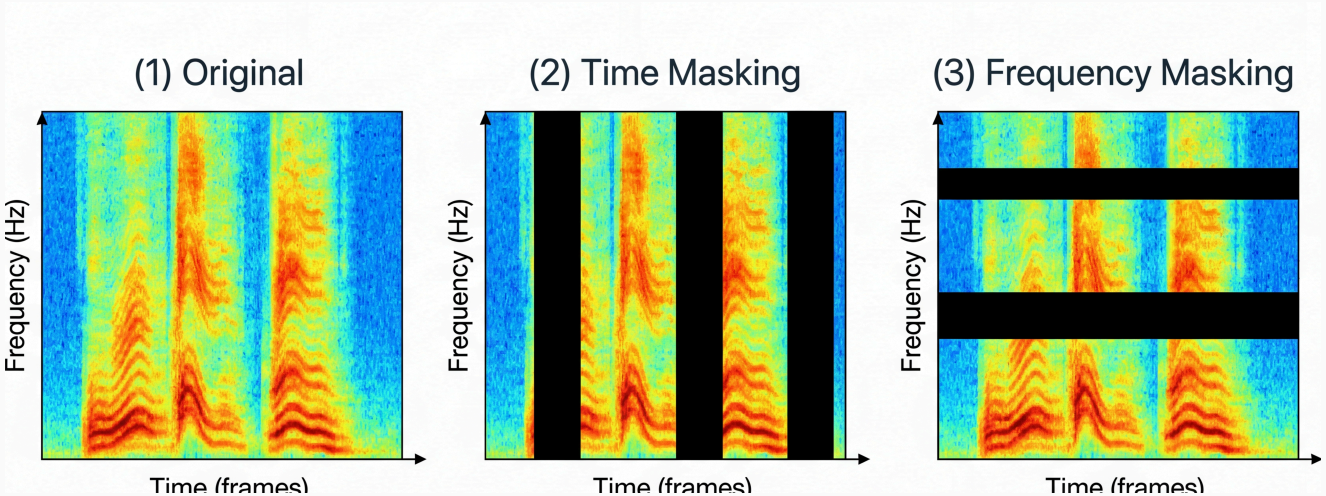
KEEP entity strings and types c_i **UPDATE** affected offsets s_i, e_i

Example adapted from STT-AI Week 5: NER-safe context editing.

For ASR, masking can keep the transcript fixed

V VISUAL

Hide some evidence, but keep the words intelligible.



input changes: $x' = \tau(x)$ **target stays:** $y' = y$

> Keep the transcript only while the masked speech is still understandable.

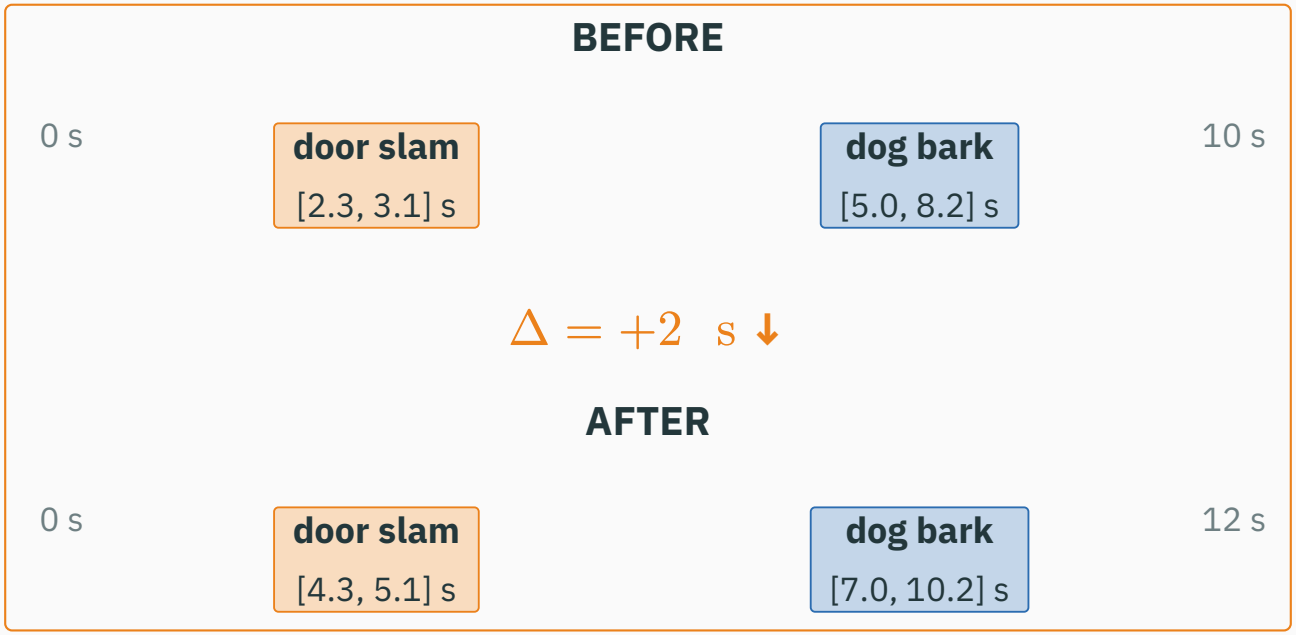
Example adapted from STT-AI Week 3: SpecAugment.

A time shift must move every event interval

V VISUAL

Shift the audio by Δ seconds; shift its timestamp labels by the same Δ.

$$[t_s, t_e]' = [t_s + \Delta, t_e + \Delta]$$



After a crop, clip intervals to the new window—or drop events that leave it entirely.
Example adapted from STT-AI Week 5: sound-event timestamps.

Transform complete samples in one PyTorch call

V VISUAL

```python
from torchvision import tv_tensors
from torchvision.transforms import v2

img = tv_tensors.Image(img)
target = {
    "boxes": tv_tensors.BoundingBoxes(
        boxes, format="XYXY",
canvas_size=img.shape[-2:]),
    "masks": tv_tensors.Mask(masks),
    "labels": labels,
}

train_aug = v2.Compose([
    v2.RandomPhotometricDistort(p=.5),
    v2.RandomIoUCrop(),
    v2.RandomHorizontalFlip(p=.5),
    v2.SanitizeBoundingBoxes(),
])

img2, target2 = train_aug(img, target)
```

One call keeps the sample synchronized

- `TVTensor` types dispatch the right transform to each target.
- Geometry shares one sampled parameter set; photometric changes touch only the image.
- `SanitizeBoundingBoxes` removes invalid boxes and matching labels or masks after a crop.

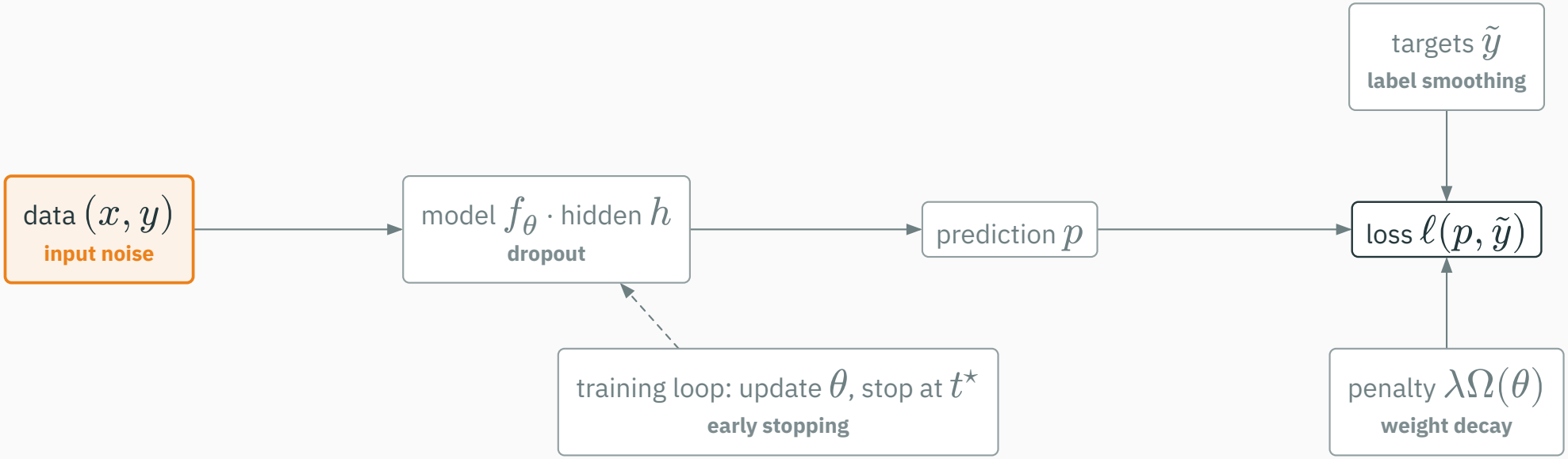
Before training

1. Overlay 32 random samples.
2. Check box canvas, positive area, and matching instance counts.
3. Check mask size and allowed IDs.
4. Unit-test one known flip numerically.

> Keep validation and test deterministic. `KeyPoints` uses the same typed-dispatch idea but remains a beta TorchVision API. official v2 detection/segmentation example

Input noise is a local kind of data augmentation

V VISUAL



Add small random nudges around each input, making sharp local changes costly; σ sets the neighbourhood size.

One example becomes a neighbourhood

V VISUAL

y_i is reused for every nearby input

x_i

fresh $x_i + \varepsilon_k$ each epoch

Each time the example is loaded:

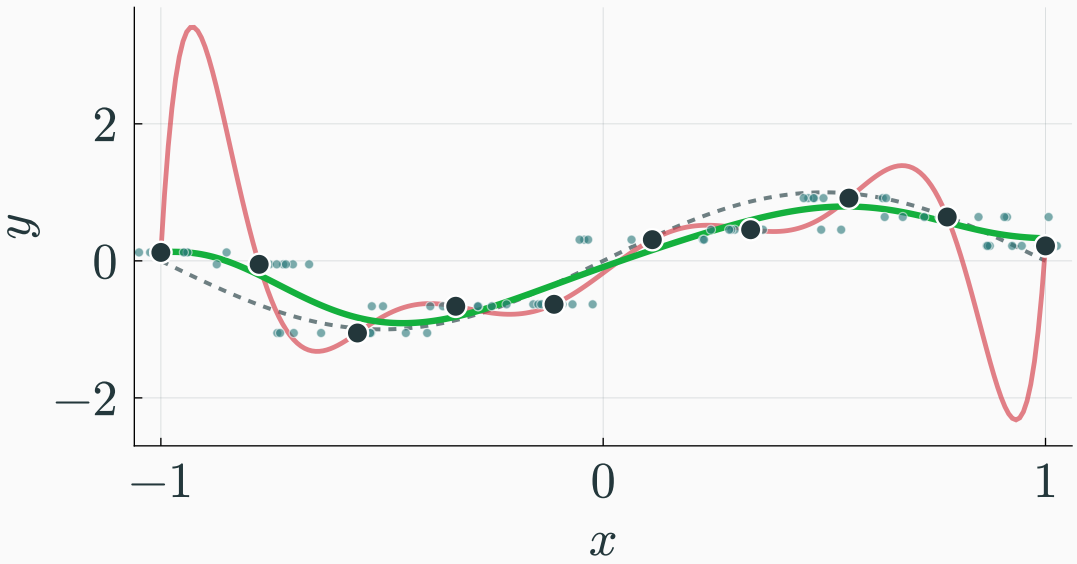
1. draw fresh noise $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$;
2. make a nearby input $x_i' = x_i + \varepsilon$;
3. train on (x_i', y_i) with the **same** target.

> The model must work around x_i, not only at the exact stored coordinate.

σ controls the radius: small enough to preserve the label, large enough to matter.

Why it helps: nearby predictions should not jump

V VISUAL



Without input noise

The red curve fits every stored point, but can swing sharply between them.

With input noise

The green curve is tested around every point, so fragile local wiggles become costly. Validation MSE falls from 1.70 to 0.048.

Why this is regularization

It prefers models whose predictions are stable near observed data, rather than models that only memorize the stored coordinates.

The noisy objective averages over nearby copies

D DERIVATION

Stored data anchors the loss. Noise draws probe nearby inputs. Validation sets σ.

$$\mathcal{L}_{\text{noise}(\theta)} = \frac{1}{n}\sum_i \mathbb{E}_{z\sim\mathcal{N}(0,I)}\left[\ell\left(f_{\theta(x_i+\sigma z)}, y_i\right)\right]$$

Examples	Noise	Selection
Average over **stored pairs** (x_i, y_i).	Average over **fresh draws** of z.	Choose σ using **validation loss**.

> For small σ, a first-order Taylor expansion gives an approximate **local-sensitivity penalty** $\left\|J_f(x)\right\|_F^2$. We add noise to the **data**. This encourages **local smoothness**.

CIFAR-10 augmentations in PyTorch

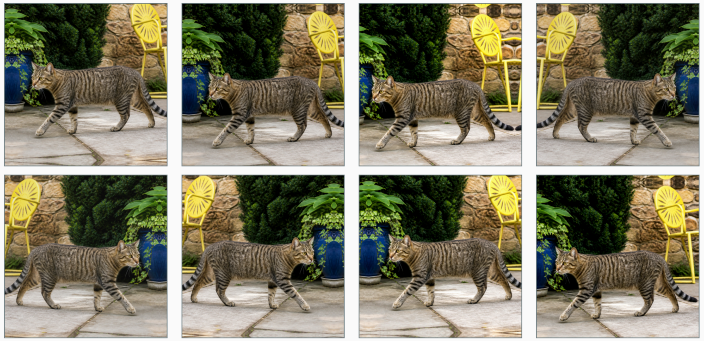
V VISUAL

```python
import torch
from torchvision.transforms import v2

train_tf = v2.Compose([
    v2.ToImage(),
    v2.RandomCrop(32, padding=4, padding_mode="reflect"),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean, std),
])

eval_tf = v2.Compose([
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean, std),
])
```

> The gallery showed the geometry at high resolution. This is the 32×32 CIFAR-10 recipe used in our controlled experiment. Training uses fresh random views, while validation and test remain deterministic. Compute `mean, std` from training only.
> torchvision v2 reference

Inspect augmented samples before training

V VISUAL



high-resolution stand-in; run the same check on your training data

For eight random views, ask:

- Is the object still present?
- Would a domain expert keep the label?
- Are boxes or masks still aligned?
- Is the strength plausible at deployment?
- Is normalization applied **after** pixel transforms?

```
import torch, torchvision

views = torch.stack([train_tf(raw_image) for _ in range(8)])
grid = torchvision.utils.make_grid(denormalize(views), nrow=4)
show(grid)                          # make this a pre-flight check
```

RandAugment has two main settings

V VISUAL

For each training image: **sample N operations** → **apply them in sequence** → **use magnitude index M**

original | `Rotate` → `Color` | `Solarize` → `TranslateX` | `Posterize` → `Contrast`

high-resolution examples from the operation pool; the sequences do not show one fixed value of M

N, `num_ops`	number of randomly chosen operations applied in sequence
M, `magnitude`	a bin index mapped to an operation-specific scale, rather than degrees or percent

RandAugment avoids a separate search for a learned policy. Validation still chooses N, M, and a task-valid operation space.

RandAugment in TorchVision

V VISUAL

```python
import torch
from torchvision.transforms import (
    InterpolationMode, v2,
)

strong_train_tf = v2.Compose([
    v2.ToImage(),  # keep uint8 here
    v2.RandomCrop(32, padding=4,
                  padding_mode="reflect"),
    v2.RandomHorizontalFlip(p=0.5),
    v2.RandAugment(
        num_ops=2,
        magnitude=9,          # bin 9 of 31
        interpolation=InterpolationMode.BILINEAR,
    ),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean, std),
])
```

What the code does

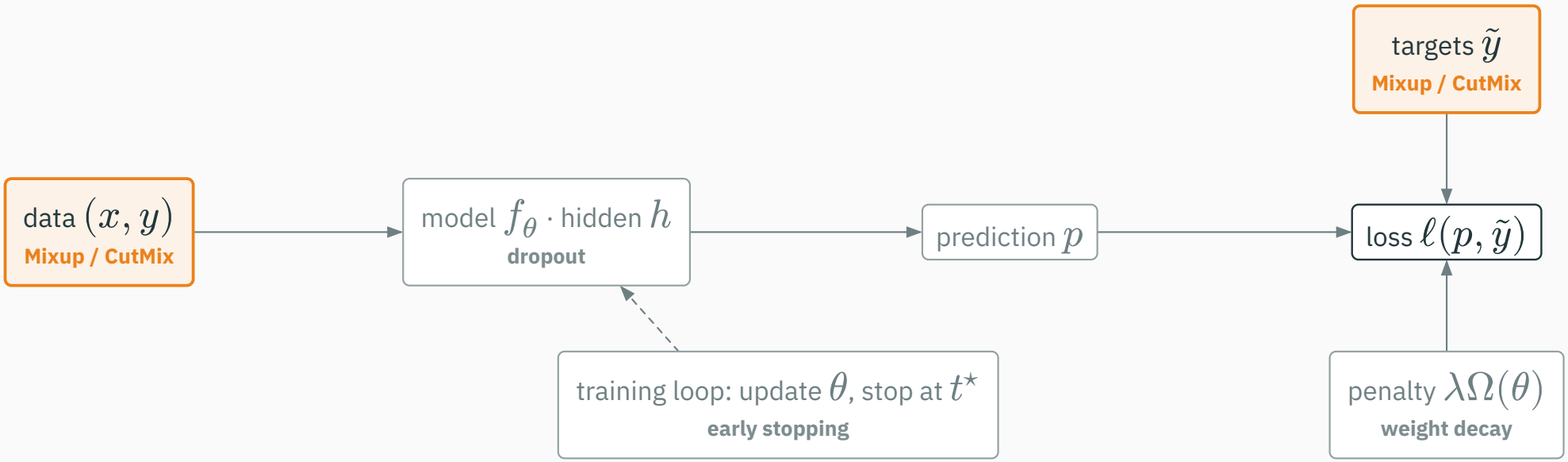
- Two operations are sampled and applied in sequence on each call.
- `magnitude=9` is a bin index, not “9 degrees.”
- Convert and normalize **after** RandAugment.
- Keep it out of validation and test.

Limitation: TorchVision’s `RandAugment` handles images and videos only. It does not update boxes, masks, or keypoints, so spatial targets need a task-aware policy.

> Horizontal flip is separate because it is not in TorchVision’s RandAugment operation pool. Inspect many draws, then validate N and M. official API

Mixup and CutMix change examples and targets

V VISUAL



INTUITION

Mix two inputs by a fraction; mix their targets by the same fraction.

WHY IT CAN HELP

Training between stored examples makes memorizing each one less useful and encourages smoother predictions.

Mixup interpolates examples and labels

D DERIVATION

$$\lambda \sim \mathrm{Beta}(\alpha, \alpha), \quad \tilde{x} = \lambda x_i + (1-\lambda) x_j, \quad \tilde{y} = \lambda e_{y_i} + (1-\lambda) e_{y_j}$$

endpoint i · cat

$\lambda = 0.70$ · soft target 70/30

endpoint j · dog

Course-generated teaching photographs with exact pixel arithmetic; the CIFAR-10 experiment uses different inputs.

The blended image need not look natural. Mixup specifies how the prediction should change between two stored examples.

For a 70/30 blend, predict 70/30

D DERIVATION

Mixup gives the blended image a **soft target**:

$$\tilde{y} = 0.70 e_{\text{cat}} + 0.30 e_{\text{dog}}$$

cat 0.70 **dog** 0.30

Cross-entropy uses those same numbers as weights:

$$\text{CE}(\tilde{y}, p) = -0.70 \log p_{\text{cat}} - 0.30 \log p_{\text{dog}}$$

> Best match: $p_{\text{cat}} = 0.70$ and $p_{\text{dog}} = 0.30$. The minimum is **positive** because neither class has target probability 1.

How strongly do examples mix?

$$\lambda \sim \text{Beta}(\alpha, \alpha)$$

- small α: λ is usually near 0 or 1;
- $\alpha = 1$: all blend ratios are equally likely;
- large α: blends stay near 50/50.

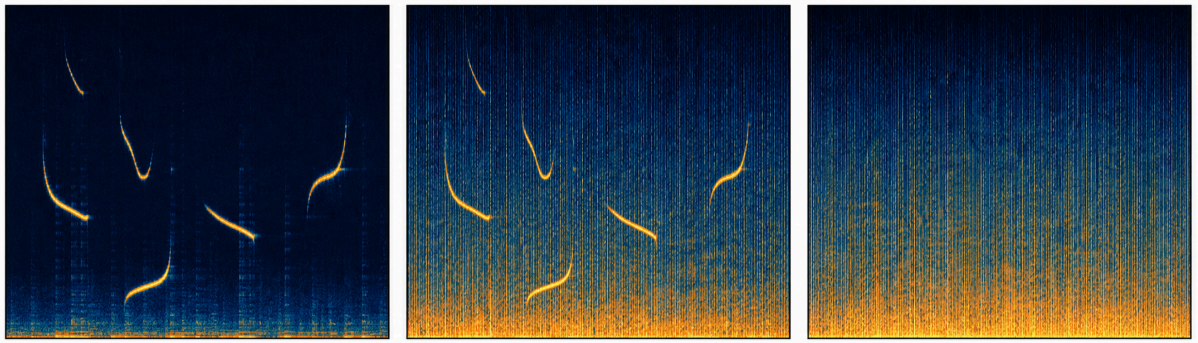
> λ weights both the pixels and the two target entries. It does not claim that the scene is literally "70% cat."

Later in the course: the entropy and KL-divergence view of cross-entropy.

Birdsong + rain can be a real recording

V VISUAL

A microphone records **sums of pressure waves**.



birdsong x_i **mixture** $\tilde{x}$ **rain** x_j

$$\tilde{x} = 0.70x_i + 0.30x_j \quad , \quad \tilde{y} = 0.70e_{\text{bird}} + 0.30e_{\text{rain}}$$

> The input mixture is physically meaningful. The target is a soft clip-level training signal that still needs validation. For multi-label detection, use multi-hot targets.

Imagen-generated schematic spectrograms; in code, mix the waveforms rather than these displayed pixels

A left turn + a right turn is not “go straight”

V VISUAL

steer $-25°$ **raw Mixup** $\rightarrow$ **target** $0°$ **steer** $+25°$

$$\tilde{x} = 0.5x_{\text{left}} + 0.5x_{\text{right}} \quad , \quad \tilde{y} = 0°$$

> The blended image is a double exposure, but the averaged target says “drive straight.” Neither source supports that instruction, so the training example is invalid.

Before using Mixup, ask three questions

D DERIVATION

1 **Is mixing meaningful in this representation?**

Often sensible: pixels, waveforms, and normalized continuous features. **Usually not:** raw token IDs, categories, graphs, and molecules.

2 **Does the mixed target match the task?**

May mix directly: global class targets. **Needs its own rule:** boxes, masks, timestamps, and actions.

3 **Does it help on untouched validation data?**

Use untouched validation: choose α on the metric that matters and inspect mixed samples for domain failures.

> If a check fails, mix later features, define the correct target rule, use a task-aware alternative, or skip mixing.

Use raw-input Mixup only when all three checks pass.

CutMix: patch-based mixing

V VISUAL

base image

25% patch from dog

Paste a rectangular region from image j into image i.

$$r = \frac{\text{patch area}}{\text{image area}}$$

$$\tilde{y} = (1 - r)e_{y_i} + re_{y_j}$$

Here $r = 0.25$, so the target is 0.75 cat + 0.25 dog.

Pixels within each pasted region remain unblended. The same domain and spatial-target checks still apply.

CutMix keeps pixels intact inside each patch while still training the prediction to reflect both source images.

Blend images and labels in interactive-articles/augmentation-mixup.

MixUp and CutMix in PyTorch

V VISUAL

```
from torch.utils.data import DataLoader
from torchvision.transforms import v2
import torch.nn.functional as F

train_loader = DataLoader(
    train_set, batch_size=B, shuffle=True)

batch_aug = v2.RandomChoice([
    v2.MixUp(num_classes=K, alpha=0.2),
    v2.CutMix(num_classes=K, alpha=1.0),
])

for images, class_ids in train_loader:
    images, soft_targets = batch_aug(
        images, class_ids
    )
    logits = model(images)
    loss = F.cross_entropy(logits, soft_targets)
    optimizer.zero_grad(set_to_none=True)
    loss.backward(); optimizer.step()
```

What changes in the training loop

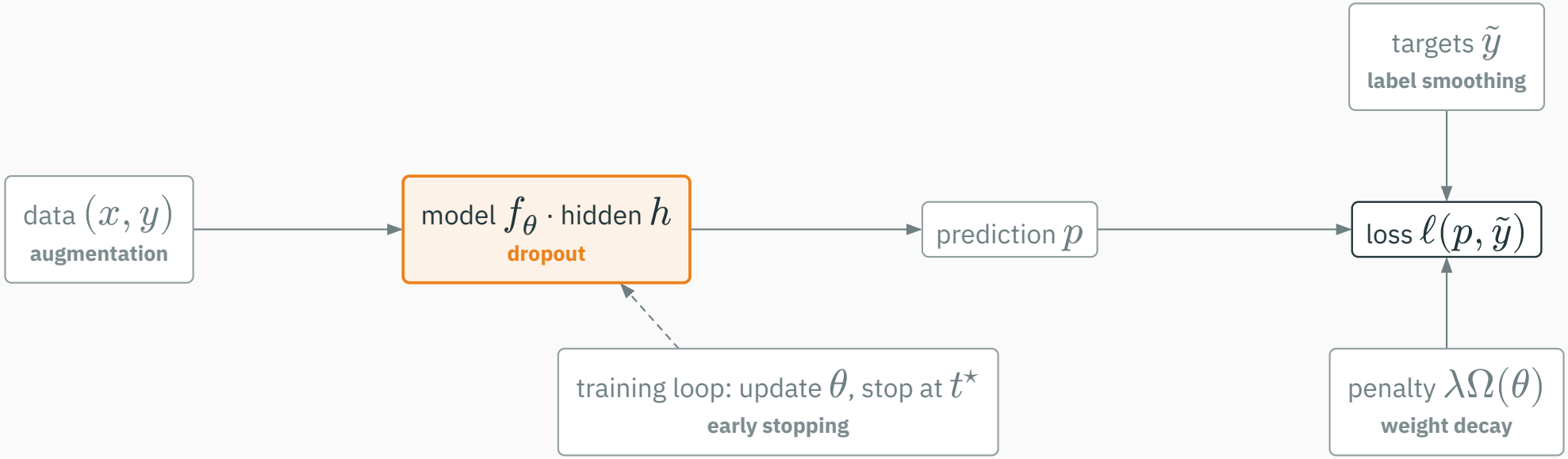
- transforms run **after** batching;
- labels change from class IDs `(B,)` to probabilities `(B, K)`;
- `cross_entropy` accepts those soft targets directly;
- the current implementation pairs adjacent samples, so shuffle the loader.

Evaluate accuracy on an unmodified deterministic split with the original class IDs.

> MixUp and CutMix run on batches rather than inside the per-image dataset pipeline. official TorchVision example

Dropout changes hidden activations during training

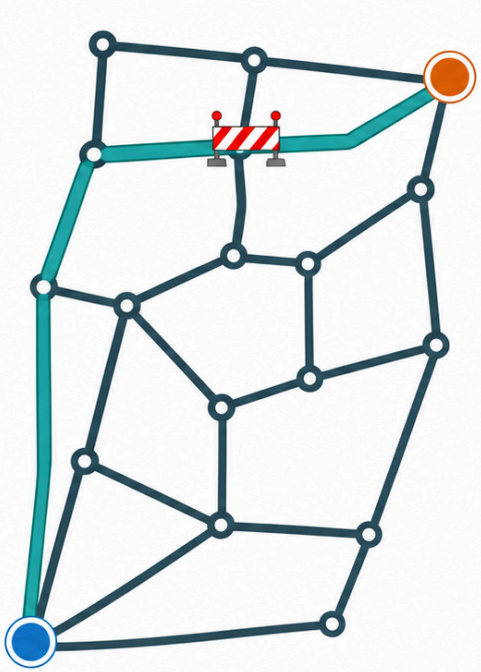
V VISUAL



Hide random hidden activations during training, so useful features cannot rely on one fixed partner; use the full network for evaluation.

One favourite route is fragile

V VISUAL



THE ROUTINE LOOKS SUCCESSFUL

Every trip follows the same familiar route, and the destination is reached.

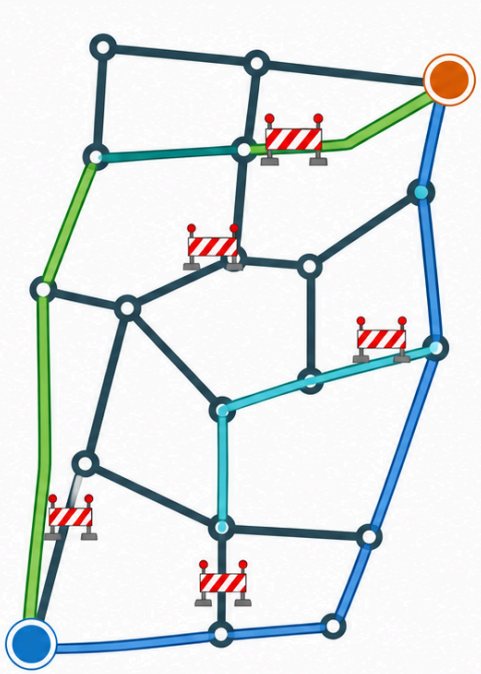
BUT THERE IS NO BACKUP

Nothing forces the traveller to learn another way. Close one critical road and the whole journey fails.

A system that depends on one pathway can handle routine cases yet remain brittle.

Practise with a different closure every trip

V VISUAL



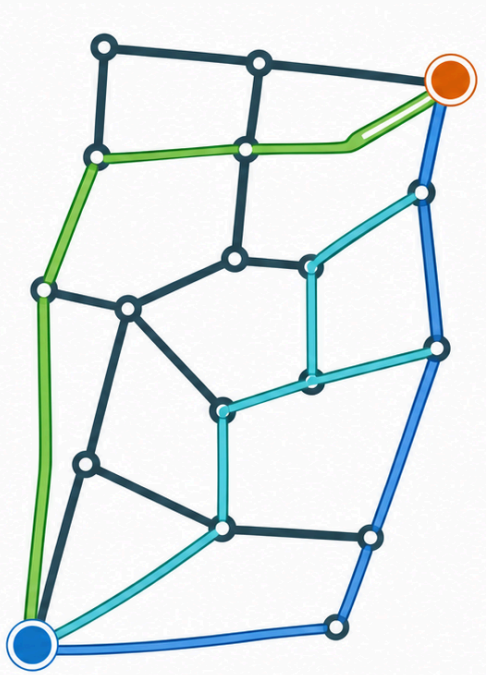
1. Close a **fresh set of roads** on each practice trip.
2. Keep the **start, destination,** and **city map** unchanged.
3. Make the traveller find a route that still works.

> A fixed closure would merely create a new favourite route. Changing the closures rewards genuine alternatives.

Random disruption changes the available route—not the task being solved.

At evaluation, open every road

V VISUAL



1. Remove the practice barriers.
2. Make **all roads** available.
3. Use the several routes learned while practising with closures.

> The traveller is not split into separate copies. One traveller learned from every disrupted trip.

Training creates backup routes; evaluation uses the complete system.

Translate the road story into dropout

V VISUAL

ROAD STORY	NEURAL NETWORK
road or route	feature pathway
temporary closure	hidden activation set to zero for **this pass**
one trip	one training forward pass
fresh closures	a fresh dropout mask
same city map	the same stored weights θ
every road open	dropout disabled for evaluation

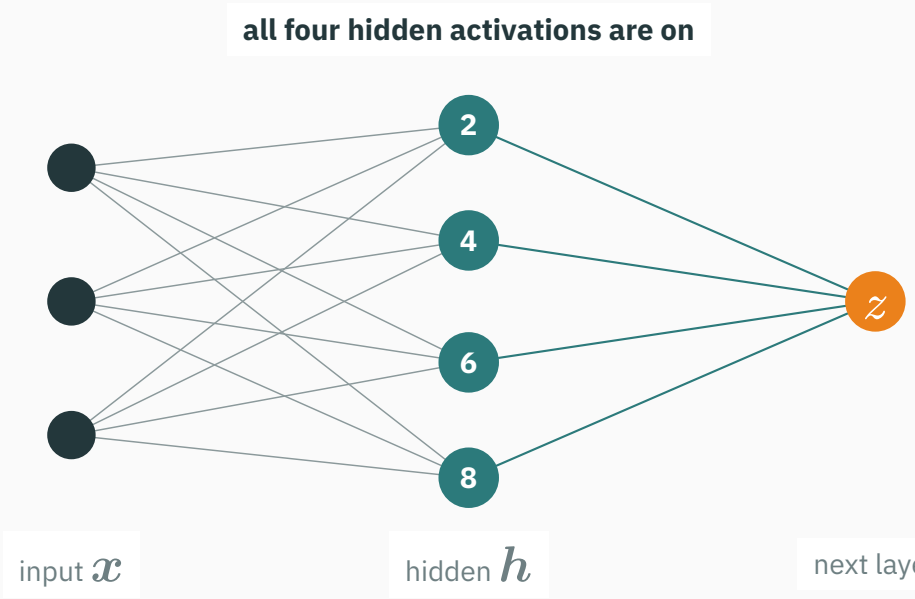
Changes for one pass: which activations are available.

Does not change: the stored network or its weights.

The same weights must succeed through many temporary subnetworks.

Back to the network: one worked example

V VISUAL



For one input, suppose the hidden layer produces

$$h = (2, 4, 6, 8)$$

Apply dropout to this hidden layer before it feeds the next layer.

> Use the **same network and same h** on every following slide.

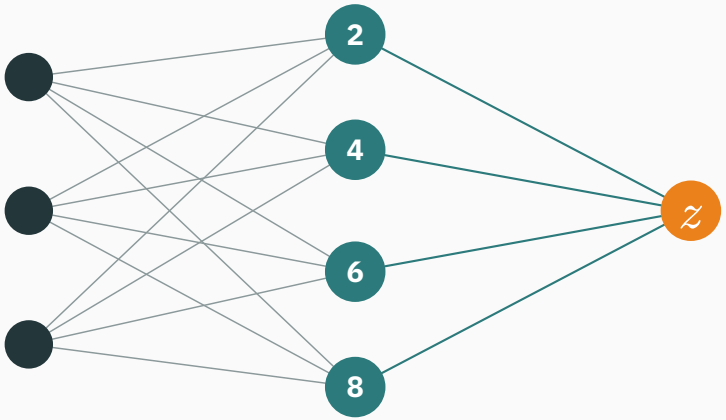
Dropout closes activations for one forward pass; it does not delete weights from the stored network.

Training pass: flip four coins

V VISUAL

BEFORE DROPOUT

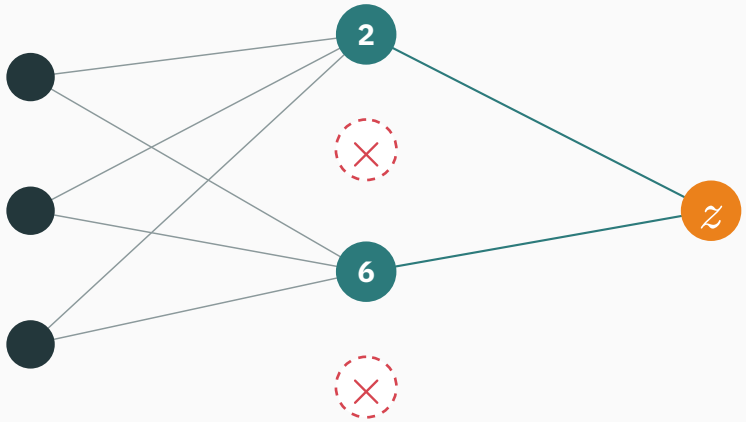
$$h = (2, 4, 6, 8)$$



input x hidden h next layer

THIS TRAINING PASS

$$m = (1, 0, 1, 0)$$



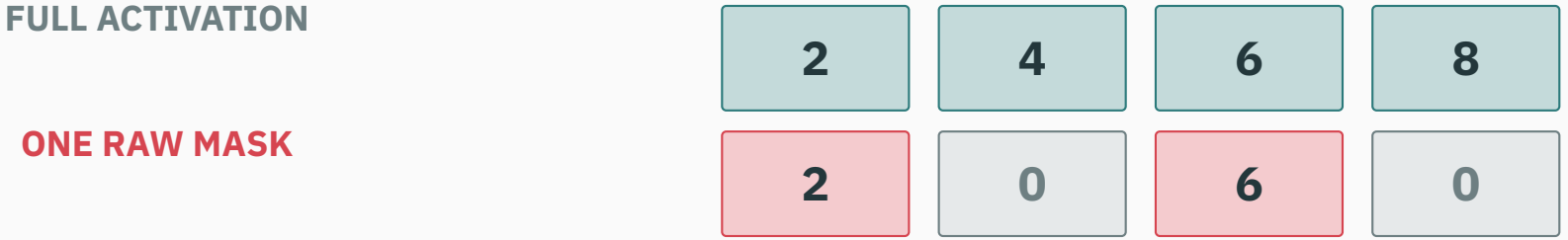
input x hidden h next layer

$$\underbrace{h}_{(2,4,6,8)} \xrightarrow{\text{mask } m} \underbrace{m \odot h}_{(2,0,6,0)}$$

First: masking lowers the average signal

V VISUAL

$q_{\text{keep}} = 0.5$ means each activation survives only half of the training passes.



ONE PASS

Some entries become zero: $(2, 4, 6, 8) \rightarrow (2, 0, 6, 0)$.

MANY PASSES

Every coordinate contributes only 50% of the time, so its average contribution is halved.

Without a correction, the next layer receives a smaller signal during training than during evaluation.

Then: scale the survivors during training

V VISUAL

If half survive on average, multiply each survivor by $2 = \frac{1}{q_{\text{keep}}}$. Dropped entries stay zero.

> The correction changes the **magnitudes** of surviving activations. It does not change which units the mask dropped. Next, zoom in on $h_3 = 6$.

One activation has two possible training-time outputs

D DERIVATION

Zoom in on $h_3 = 6$, with $q_{\text{keep}} = 0.5$. A fresh mask is drawn on every forward pass.

KEPT	DROPPED
This happens with probability 0.5.	This happens with probability 0.5.
$\frac{6}{0.5} = 12$	0
The next layer receives **12**.	The next layer receives **0**.

One pass sends either 12 or 0—not 6. The 6 reappears only after averaging over masks.

Expected value is the long-run average over masks

D DERIVATION

Repeat the pass many times: about half the masks send 12, and half send 0.

$$\mathbb{E}\left[\tilde{h}_3\right] = 0.5 \cdot 12 + 0.5 \cdot 0 = 6 = h_3$$

KEPT CONTRIBUTION	DROPPED CONTRIBUTION	LONG-RUN AVERAGE
probability kept $\times$ value when kept	probability dropped $\times$ value when dropped	the value received on average

“Expected” does not mean every pass returns 6. The random training outputs are centred on 6.

The general rule has the same two branches

D DERIVATION

Now replace 6 and 0.5 with any activation h_i and keep probability q_{keep}.

KEPT BRANCH	DROPPED BRANCH
probability q_{keep}	probability $1 - q_{\text{keep}}$
value $\frac{h_i}{q_{\text{keep}}}$	value 0

$$\mathbb{E}\left[\tilde{h}_i\right] = q_{\text{keep}}\left(\frac{h_i}{q_{\text{keep}}}\right) + \left(1 - q_{\text{keep}}\right) \cdot 0$$

Now simplify one coloured contribution at a time.

The two contributions simplify separately

D DERIVATION

KEPT CONTRIBUTION

$$q_{\text{keep}}\left(\frac{h_i}{q_{\text{keep}}}\right) = h_i$$

The keep probability cancels the factor $\frac{1}{q_{\text{keep}}}$.

DROPPED CONTRIBUTION

$$\left(1 - q_{\text{keep}}\right) \cdot 0 = 0$$

Anything multiplied by the dropped value 0 contributes 0.

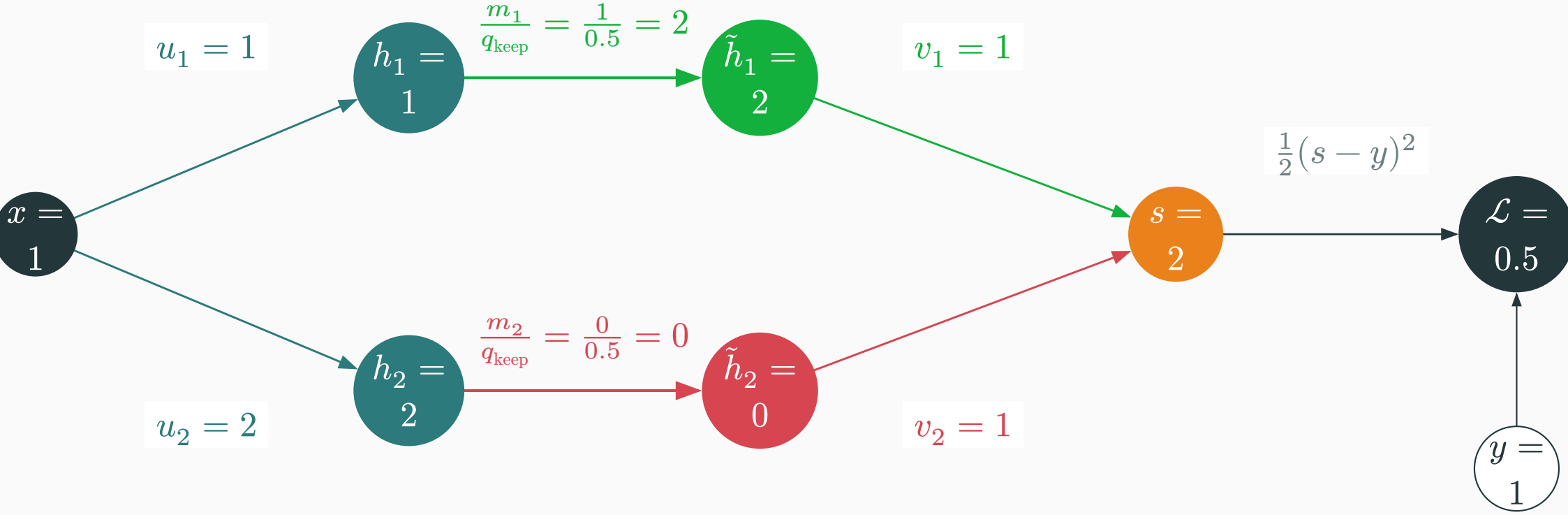
$$\mathbb{E}\left[\tilde{h}_i\right] = h_i + 0 = h_i$$

Training is noisy, but its mean activation matches the value used at evaluation.

Forward: one path survives, one is dropped

V VISUAL

Tiny linear network, no biases: train $u = (1, 2)$ and $v = (1, 1)$. For this pass, $q_{\text{keep}} = 0.5$ and $m = (1, 0)$ are constants in $\tilde{h}_i = \left(\frac{m_i}{q_{\text{keep}}}\right) h_i$.



Start backward at the loss

D DERIVATION

First compute the gradient arriving at the output of the dropout layer.

$$g_s = \frac{\partial \mathcal{L}}{\partial s} = s - y = 2 - 1 = 1$$

OUTGOING WEIGHTS

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{v}} = g_s \tilde{\boldsymbol{h}} = (2, 0)$$

v_1 **gets gradient** 2**.**
v_2 **gets gradient** 0 because its input was zero.

GRADIENT AT DROPOUT OUTPUT

$$\frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{h}}} = g_s \boldsymbol{v} = (1, 1)$$

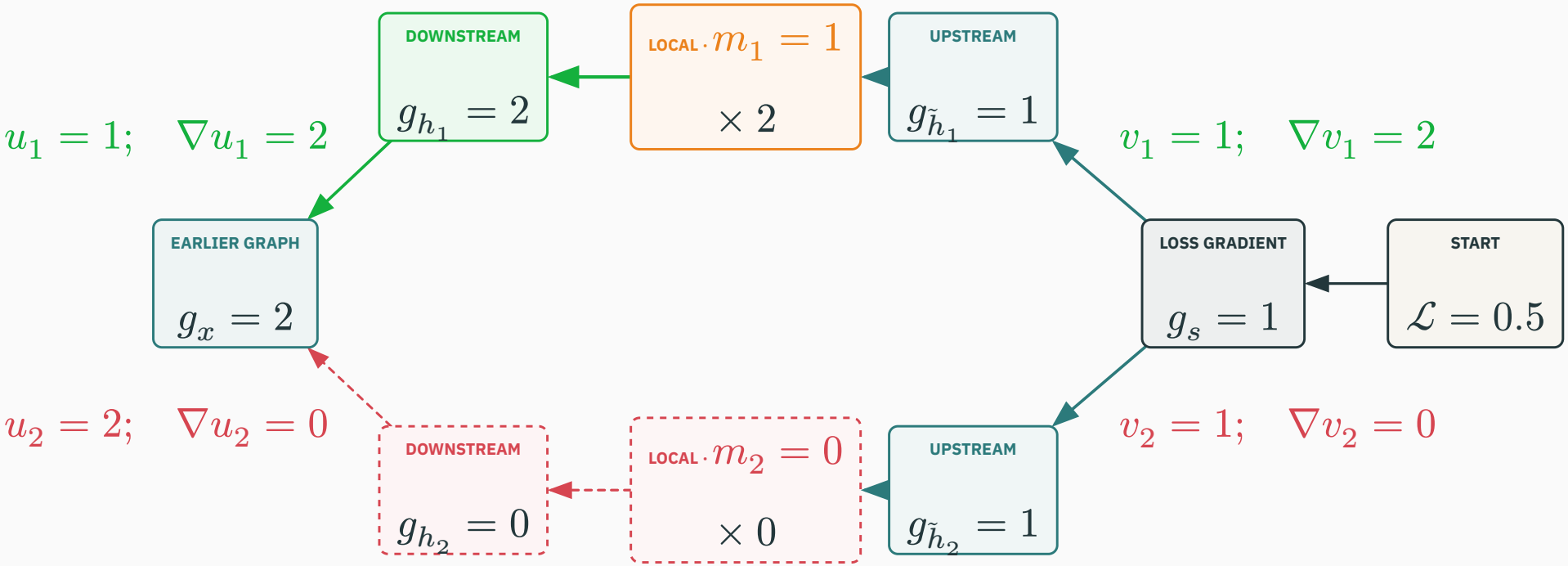
Both output slots receive gradient **before** it crosses the dropout gate.

The dropped path becomes zero in the next chain-rule step—not before it.

Backward: upstream × local = downstream

D DERIVATION

Backward reuses the mask: $\frac{\partial \tilde{h}_i}{\partial h_i} = \frac{m_i}{q_{\text{keep}}}$. Here $q_{\text{keep}} = 0.5$, so the local factors are 2 and 0.



Complete the end-to-end gradients

D DERIVATION

Now follow that gradient to the weights before and after each hidden node.

INCOMING WEIGHTS

Since $\boldsymbol{h} = \boldsymbol{u}x$ and $x = 1$,

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}} x = (2, 0)$$

u_1 gets gradient 2.
u_2 gets gradient 0 from this example.

OUTGOING WEIGHTS

The earlier step gave

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{v}} = (2, 0)$$

v_1 gets gradient 2.
v_2 gets gradient 0 from this example.

One vanilla-SGD step completes the example

D DERIVATION

For this one-example step there is no optimizer state or decay, so zero gradient means no motion.

PARAMETER UPDATE · $\eta = 0.1$

$$\theta \to \theta - \eta \nabla \theta$$

parameter	before	gradient	after
u_1	1	2	0.8
u_2	2	0	2
v_1	1	2	0.8
v_2	1	0	1

A fresh mask may reverse the roles next pass.

GRADIENT TO THE EARLIER GRAPH

$$\frac{\partial \mathcal{L}}{\partial x} = u_1 \frac{\partial \mathcal{L}}{\partial h_1} + u_2 \frac{\partial \mathcal{L}}{\partial h_2} = 1 \cdot 2 + 2 \cdot 0 = 2$$

Gradient still reaches x through the kept route.

Real training: other batch examples, optimizer state, or weight decay can still move a parameter. Dropout gates only this example's data-loss gradient.

Evaluation: the same network, all units on

V VISUAL

TRAINING

draw a fresh $\boldsymbol{m}$, then send $\tilde{\boldsymbol{h}} = \frac{\boldsymbol{m} \odot \boldsymbol{h}}{q_{\text{keep}}}$ to the next layer.

`model.train()`

EVALUATION

use the full activation $\boldsymbol{h} = (2, 4, 6, 8)$ once—no mask and no extra scaling.

`model.eval()`

The $\frac{1}{q_{\text{keep}}}$ correction happens during training, so evaluation is simply the identity: $h \mapsto h$.

Why dropout regularizes: distribute the evidence

D DERIVATION

BRITTLE RELIANCE

$$w = (2, 0, 0, 0)$$

One feature carries everything. If it is masked, the unit loses all evidence.

$$\|w\|_2^2 = 4$$

DISTRIBUTED RELIANCE

$$w = (0.5, 0.5, 0.5, 0.5)$$

Several features can support the unit when a partner disappears.

$$\|w\|_2^2 = 1$$

Any input may vanish $\rightarrow$ concentrating all influence in one pathway becomes risky.

> This is an L_2**-like** pressure, not the same objective as ordinary weight decay. The effective penalty depends on the data and activation scales; the equal weights above are an illustration, not a theorem.

What to do in PyTorch

V VISUAL

```
model = nn.Sequential(
    nn.Linear(d, 256),
    nn.ReLU(),
    nn.Dropout(p=0.2),   # drop 20%; keep q=0.8
    nn.Linear(256, K),
)

model.train()   # fresh masks: ON
# ... training loop ...

model.eval()    # dropout: OFF
# ... validation / test ...
```

PLACE IT

After selected hidden activations—not on final logits.

START MODESTLY

Try $p_{\text{drop}} \approx 0.1\text{–}0.3$ only when validation shows overfitting.

LET PYTORCH SCALE

`nn.Dropout` performs inverted scaling automatically. Do not divide manually.

PyTorch's `p` means drop probability. Some implementations instead use `keep_prob` = $q_{\text{keep}} = 1 - p$**.**

What does dropout return in evaluation mode?

Q QUESTION

PyTorch's argument `p` is p_{drop}, so $q_{\text{keep}} = 1 - p_{\text{drop}}$.

```
h = torch.tensor([2., 4., 6., 8.])
drop = nn.Dropout(p=0.5)
drop.train(); drop(h)   # one draw gave: tensor([ 4.,  0., 12.,  0.])
drop.eval();  drop(h)   # = ?
```

What does the `eval()` line return?

A. $(2, 4, 6, 8)$, the identity

B. $(1, 2, 3, 4)$, scaled by q_{keep}

C. $(4, 8, 12, 16)$, scaled by $1/q_{\text{keep}}$

D. a fresh random mask, like training

Answer: dropout is disabled during evaluation

A ANSWER

Correct: A — the identity, because dropout is disabled at test time

Why: Inverted dropout applies the $1/q_{\text{keep}}$ correction during training. Evaluation therefore needs no rescaling and is deterministic.

A model left in `train()` mode during evaluation gives noisy, degraded predictions. Call `model.eval()` before computing validation or test metrics.

`train()`, `eval()`, and `inference_mode()`

V VISUAL

control	dropout	BatchNorm	autograd
`model.train()`	random masks	batch statistics; buffers update	unchanged
`model.eval()`	identity	stored statistics	unchanged
`torch.inference_mode()`	unchanged	unchanged	disabled

`eval()` changes module behaviour. `inference_mode()` changes gradient tracking.

Dropout in modern architectures

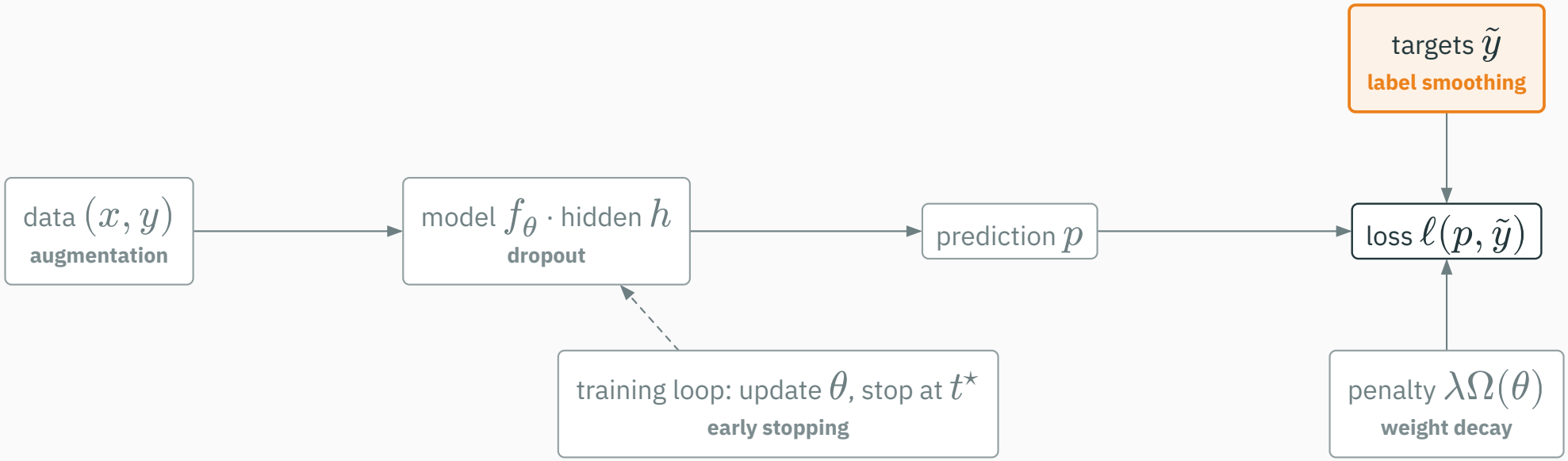
V VISUAL

setting	current practice
wide fully-connected heads	a common location because co-adaptation is strongest there
transformer blocks	small rates inside attention / FFN and on residuals
modern conv backbones	often secondary to strong augmentation and weight decay; classifier-head dropout remains common
whole residual branches	the same idea at branch scale: **stochastic depth / DropPath**

> Tune p_{drop} on **validation**, just as you tune λ. A small value may have little effect, while a large value can destroy useful signal.

Label smoothing changes the training targets

V VISUAL



Replace one-hot targets with slightly softer targets, so extreme confidence is no longer rewarded; validation chooses ε.

Why one-hot cross-entropy keeps widening the gap

D DERIVATION

Getting the class right is not the finish line: a one-hot target still rewards more certainty.

1 · ONE-HOT LOSS

Suppose the correct class is “b”:

$$\boldsymbol{y} = (0, 1, 0, 0, 0)$$

$$\ell = -\sum_k y_k \log p_k = -\log p_b$$

Every wrong-class term disappears because its target weight is 0.

2 · LOGIT GRADIENT

Cross-entropy changes the logits through

$$\frac{\partial \ell}{\partial z_k} = p_k - y_k$$

$p_b - 1 < 0 \rightarrow$ gradient descent pushes z_b **up**.

$p_j - 0 > 0 \rightarrow$ it pushes every wrong z_j **down**.

In a symmetric score picture: $z_b \rightarrow +\infty \qquad z_j \rightarrow -\infty$

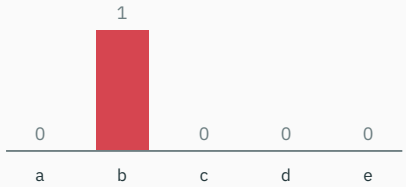
What matters: every gap $z_b - z_j$ has no finite stopping point.

Label smoothing says “confident enough”

D DERIVATION

Ask for high confidence, not absolute certainty.

ONE-HOT



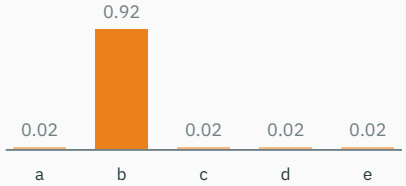
$$y = (0, 1, 0, 0, 0)$$

$$\frac{\partial \ell}{\partial z_b} = p_b - 1$$

The push stops only as $p_b \to 1$: no finite logit gap is enough.

SMOOTHED

$$\tilde{y} = (1 - \varepsilon)y + \frac{\varepsilon}{K}\mathbf{1}$$



$$\tilde{y} = (.02, .92, .02, .02, .02)$$

$$\frac{\partial \ell}{\partial z_b} = p_b - 0.92$$

The push stops at $p_b = 0.92$. This corresponds to the finite correct-vs-wrong logit gap $z_b - z_j = \ln\left(\frac{0.92}{0.02}\right) \approx 3.83$.

Use label smoothing modestly—and validate it

V VISUAL

PYTORCH

```
epsilon = 0.1
criterion = nn.CrossEntropyLoss(
    label_smoothing=epsilon
)
loss = criterion(logits, class_ids)
```

Train with smoothed targets; keep validation and test labels hard.

The smoothed training loss has a positive floor, so do not compare it directly with a one-hot run.

CAN HELP

- reduce pressure toward extreme logits;
- improve validation accuracy, NLL, or calibration in some recipes.

COSTS AND LIMITS

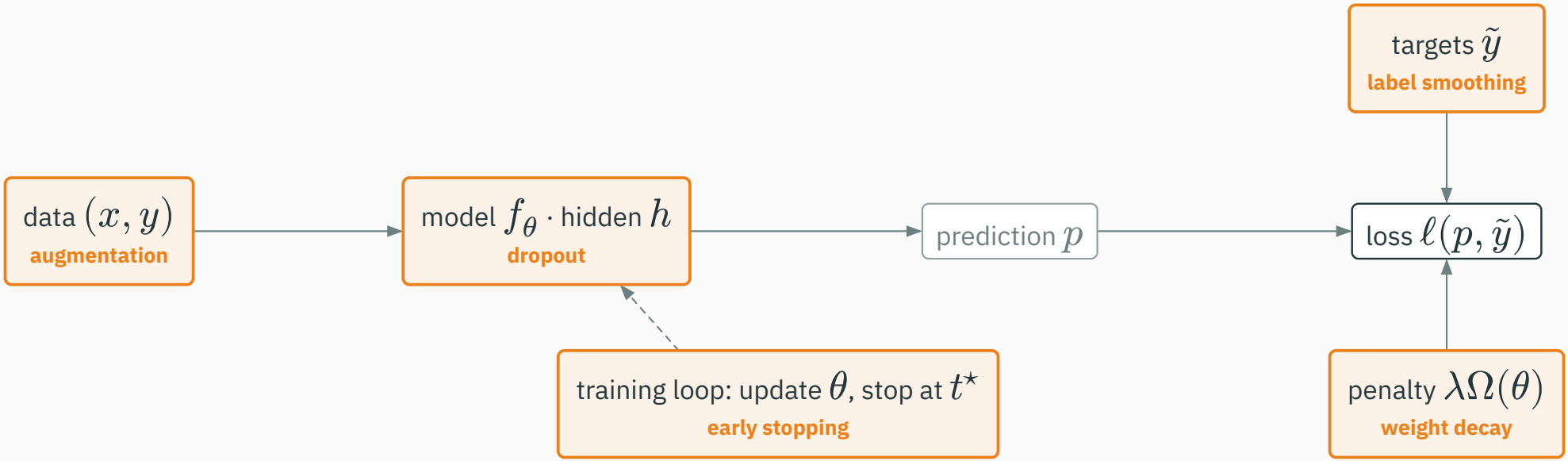
- pulls every prediction toward uniformity;
- does not know that “wolf” is closer to “husky” than “airplane”;
- may over-soften targets on top of Mixup or CutMix.

> This is a regularizing pressure, not a promise. Start small, tune ε on untouched **validation**, and—if Mixup or CutMix is active—ablate smoothing separately.

Regularization in practice

Regularization cheat sheet: five places to intervene

V VISUAL



Diagnose first. Change one site at a time, then let validation choose the strength.

Mixup and CutMix span both the data and target sites.

A taxonomy of regularization methods

V VISUAL

site	methods	knob
objective	weight decay (L_2), L_1, spectral penalties	λ
training loop	early stopping, minibatch noise	$t^\star$, batch size
model / features	dropout, stochastic depth	p
data	crops · flips · Gaussian input noise, Mixup, CutMix	strength, σ, α
targets	label smoothing	ε
architecture	convolution, weight sharing, bottlenecks	design choice

> Architecture adds no explicit penalty term, but it still restricts the function family. Treat it as a design choice selected by validation.

Implicit regularization in training

V VISUAL

implicit source	selection pressure it exerts
SGD minibatch noise	a stochastic trajectory with optimizer- and batch-size-dependent implicit bias
initialization + early phase	which basin training explores first (Lecture 6: initialization)
architecture	some functions are easy to express, others practically unreachable
BatchNorm's batch statistics	per-batch jitter in activations as a **side effect**

> Treat BatchNorm primarily as an **optimization** method (Lecture 6). It improves trainability, with mild regularization as a side effect. Changing the optimizer or batch size can also change the model's **effective** regularization.

A practical regularization workflow

V VISUAL

fixed baseline → valid augmentation → weight decay → one extra method

1	baseline	log train/validation curves; restore the best checkpoint
2	augmentation	add domain-valid transforms; inspect actual batches
3	weight decay	sweep λ on a log scale; compare at the same compute budget
4	other methods	if needed, ablate dropout, MixUp/CutMix, or label smoothing one at a time

Specialized regularization methods

V VISUAL

These methods address more specific problems:

stochastic depth / DropPath	dropout at residual-branch scale; standard in deep ViTs
spectral normalization	constrain layer operator norms (GANs, sensitivity control)
SAM	optimize the worst loss in a parameter neighbourhood
adversarial training	augment with worst-case perturbations to improve robustness, at a cost

> Change one factor at a time so the comparison remains interpretable. Evaluate the sealed test set once, after every knob is fixed.

Diagnosing common failure modes

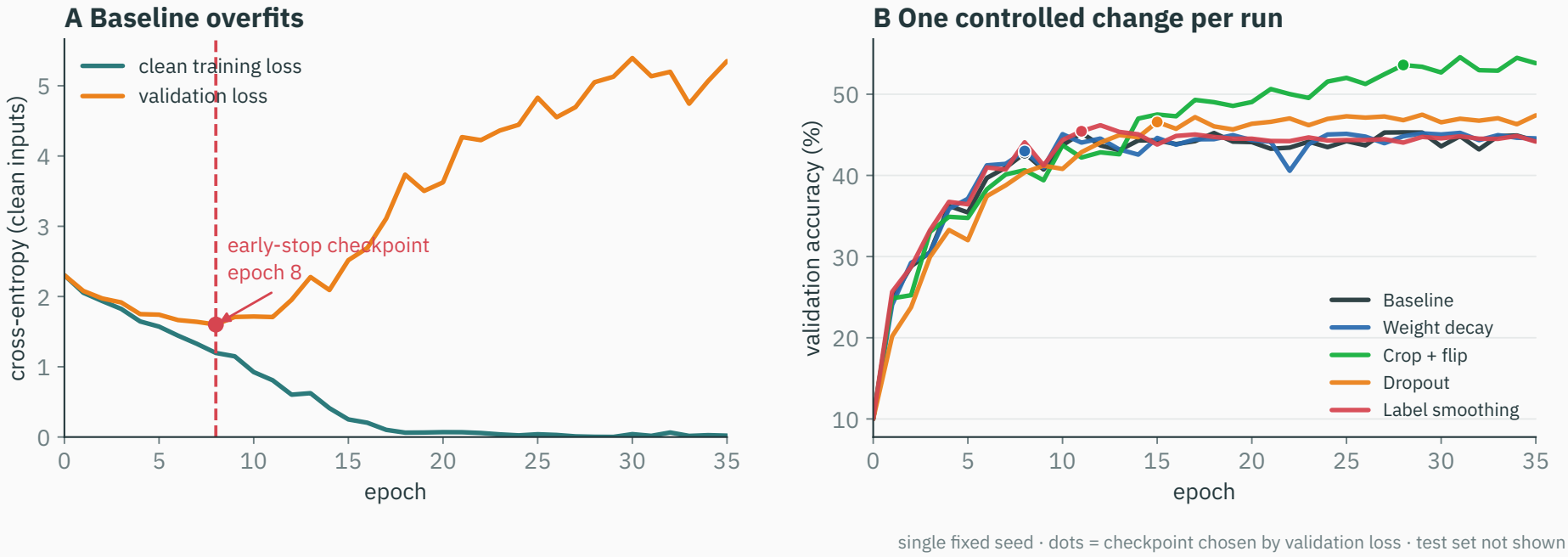
V VISUAL

symptom	first suspect	controlled test
train and val both high	underfit / optimization	fix fit first (L5 to L6); regularizing makes it **worse**
train ↓, val turns ↑	overfitting over time	restore best checkpoint; then sweep one knob
val **better** than train	train-only noise: dropout, aug, penalty in train loss	re-measure both in `eval()` mode, penalty excluded
great val, poor deployment	validation no longer represents reality	audit the split and the data pipeline, not the knobs
confident and wrong	target pressure	add smoothing; **measure** calibration

Use regularization after diagnosing the failure mode, and choose its strength by validation. Do not add it by default or choose an arbitrary strength.

Baseline overfitting on CIFAR-10

V VISUAL

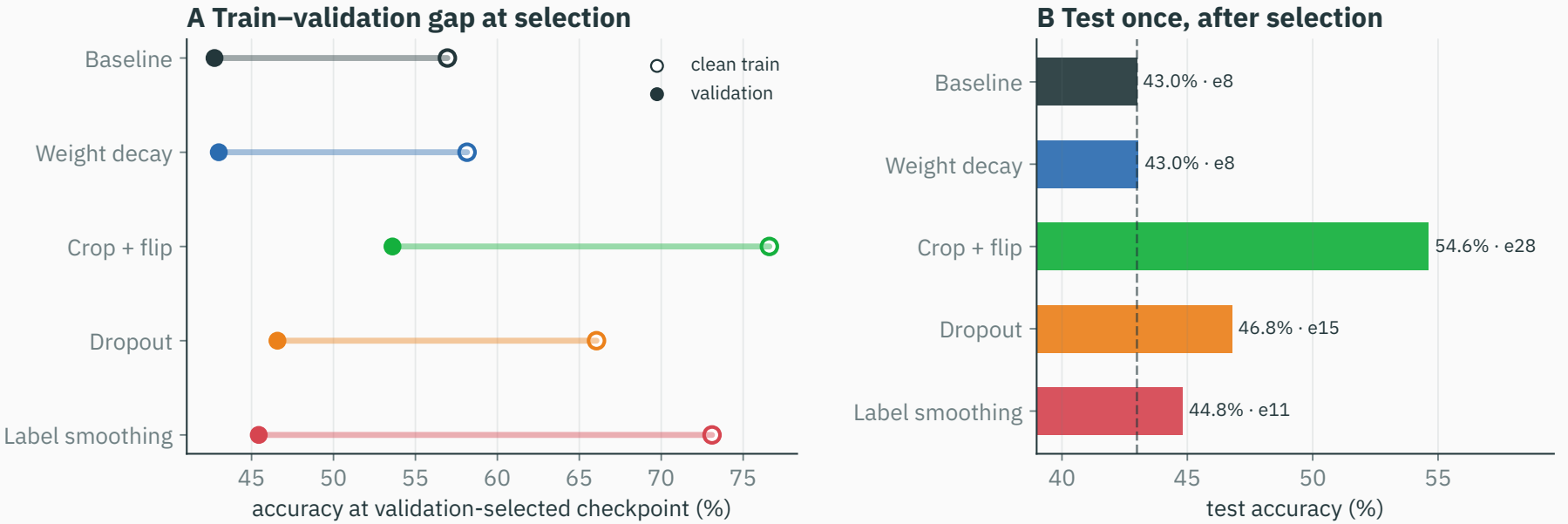


single fixed seed · dots = checkpoint chosen by validation loss · test set not shown

> The run uses 2,000 stratified training examples, one fixed initialization and minibatch order, the same 666,538-parameter CNN, and a 35-epoch budget. Dots mark the checkpoint selected by validation loss.

Controlled CIFAR-10 ablations

V VISUAL

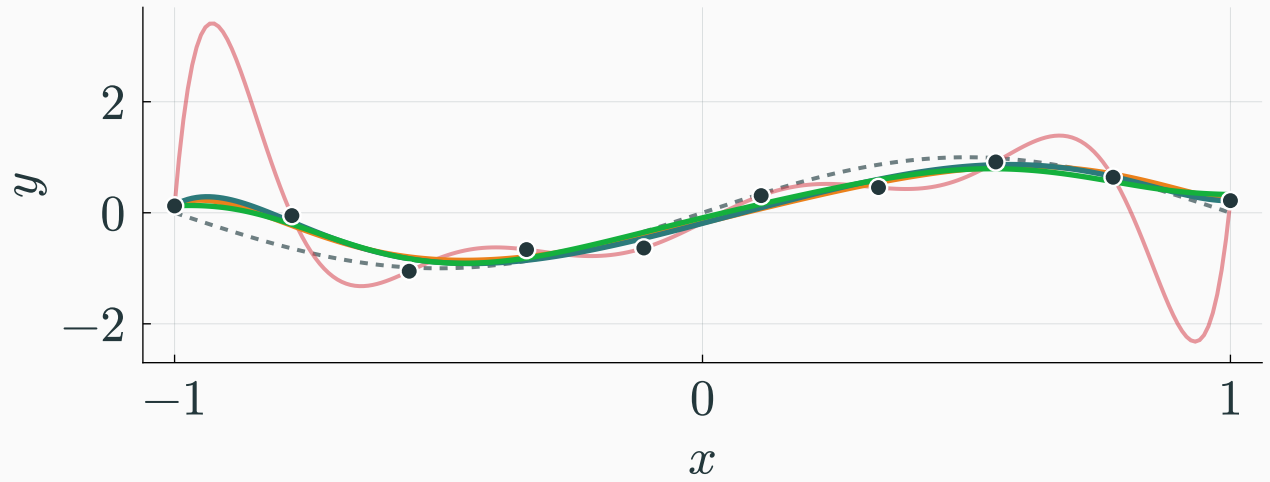


2,000 train · 5,000 validation · 10,000 test · one seed (descriptive, not a benchmark)

> Crop + flip performs best; dropout and smoothing help modestly; the tested weight-decay setting ($\lambda_{\text{wd}} = 0.01$) is neutral. Choices were predeclared and checkpoints used validation; the one-seed test result is not a benchmark.

Three regularizers on the polynomial example

V VISUAL



Weight decay, early stopping, and input jitter act at different sites and use different knobs. Here they favour a less variable fit and produce nearly identical curves.

Final model selection and test evaluation

V VISUAL

model	choice made without test	val MSE	test MSE
interpolant	unregularized reference	1.704	1.187
weight decay	$\lambda^\star = 0.001$ from validation	0.065	0.124
early stop	$t^\star = 5000$ from validation	0.061	0.125
input jitter	policy fixed before test	0.048	0.113

> Use the test column only for final reporting. If it changes a knob, the test set has become another validation set.

SYNTHETIC · COMPUTED: seeded 10-point teaching case. Every curve and number on this slide is recomputed at compile time.

Before you regularize

1. Which learning curves did you plot, and what do they diagnose?

2. Which single site are you intervening on, and with which knob?

3. What did validation choose? Did training loss influence the choice?

4. Has the test set remained untouched for one final evaluation?

5. Which implicit regularizers moved when you changed optimizer or batch size?

Reproduce: log the seed, splits, schedule, and every knob.

If you have not plotted learning curves, diagnose the problem before choosing a regularizer.

Primary sources

- Zhang, Bengio, Hardt, Recht, Vinyals (2017), **Understanding deep learning requires rethinking generalization**. Shows that networks can fit random labels.
- Srivastava, Hinton, Krizhevsky, Sutskever, Salakhutdinov (2014), **Dropout**.
- Zhang, Cissé, Dauphin, Lopez-Paz (2018), **mixup** · Yun et al. (2019), **CutMix** · Cubuk et al. (2020), **RandAugment**.
- Szegedy et al. (2016), label smoothing · Müller, Kornblith, Hinton (2019), **When Does Label Smoothing Help?**.
- Bishop (1995), **Training with noise is equivalent to Tikhonov regularization**; Belkin et al. (2019), double descent the modern interpolation picture.

> The seeded polynomial figures are recomputed at compile time. The opening schematic, CIFAR panels, and controlled PyTorch ablation have documented provenance and scripts in `lecture7/figures/PROVENANCE.md`. The companion Typst lab independently checks the regression numbers.

Regularization supplies a preference where the training data do not determine the model’s behaviour. Validation chooses its strength.

Diagnose with learning curves. Change one factor, then measure again on validation.

Optional: AdamW

Adam and AdamW treat decay differently

★ OPTIONAL

Let $\mathcal{A}_t(u)$ denote Adam's update computed from u using the current moment state.

optimizer	parameter update	what happens to decay
plain SGD	$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \boldsymbol{g}_t$	none
SGD + L_2	$\boldsymbol{\theta}_{t+1} = (1 - \eta\lambda)\boldsymbol{\theta}_t - \eta \boldsymbol{g}_t$	proportional shrink
Adam + L_2	$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathcal{A}_t(\boldsymbol{g}_t + \lambda\boldsymbol{\theta}_t)$	penalty enters Adam's moments
AdamW	$\boldsymbol{\theta}_{t+1} = (1 - \eta\lambda)\boldsymbol{\theta}_t - \eta \mathcal{A}_t(\boldsymbol{g}_t)$	shrink stays separate

> For plain SGD, L_2 and weight decay are equivalent. With Adam, coupled L_2 enters the moment estimates; AdamW keeps proportional shrinkage separate.
> Loshchilov & Hutter (2019)

With Adam, use AdamW when you want weight decay to remain a separate proportional shrinkage step.

PyTorch: AdamW

★ OPTIONAL

```
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
```

> `weight_decay` is λ. Each step applies $\boldsymbol{\theta} \leftarrow (1 - \eta_t \lambda)\boldsymbol{\theta}$ before Adam's data update. Do not also add $(\lambda/2)\|\theta\|_2^2$ to the loss. This call decays every parameter; use parameter groups to exempt bias or normalization parameters.